



**POLITECHNIKA
RZESZOWSKA**
im. IGNACEGO ŁUKASIEWICZA



**WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ**
POLITECHNIKI RZESZOWSKIEJ

POLITECHNIKA RZESZOWSKA
im. Ignacego Łukasiewicza
WYDZIAŁ MATEMATYKI I FIZYKI STOSOWANEJ

Dawid Stachiewicz

**OPTYMALIZACJA SKUTECZNOŚCI
SYSTEMU IDS W WYKRYWANIU
ATAKÓW DDoS**

Promotor: Dr inż. Mariusz Nycz

PRACA INŻYNIERSKA

kierunek studiów: Inżynieria i Analiza Danych

Rzeszów 2025

Spis treści

Wykaz symboli, oznaczeń i skrótów	5
Wstęp	7
1 Wprowadzenie do teorii systemów detekcji intruzów	9
1.1. Systemy wykrywania wtargnięć (IDS) - charakterystyka	9
1.2. Klasyfikacja IDS: NIDS vs HIDS	10
1.3. Metody detekcji: sygnaturowa, anomalii, hybrydowa	10
1.4. Ataki DDoS - istota, typy i mechanizmy	11
1.5. Metryki skuteczności IDS	12
1.6. Wyzwania optymalizacji IDS w środowiskach korporacyjnych	13
1.6.1. Wydajność i przetwarzanie w czasie rzeczywistym	13
1.6.2. Problem nadmiarowości alarmów	14
1.6.3. Złożoność zarządzania regułami	14
2 Opis eksperymentów i metody analizy wyników	15
2.1. Cel, zakres i założenia badań	15
2.2. Opis środowiska testowego	16
2.2.1. Struktura i oprogramowanie	16
2.2.2. Organizacja sieci i topologia	16
2.3. Konfiguracja systemów IDS	17
2.3.1. Snort	17
2.3.2. Suricata	18
2.3.3. Tryb hybrydowy (Korelacja)	18
2.4. Scenariusze i procedura testowa	19
2.4.1. Rodzaje ataków	19
2.4.2. Poziomy intensywności	19
2.4.3. Automatyzacja procesu	20
2.5. Zbieranie i analiza danych	20
2.6. Metodyka oceny skuteczności	20
2.7. Walidacja i ograniczenia metodyki	21

3	Implementacja środowiska badawczego i przebieg eksperymentów	23
3.1.	Konfiguracja maszyn wirtualnych i infrastruktury testowej	23
3.2.	Wdrożenie i konfiguracja systemu Snort	23
3.2.1.	Implementacja reguł detekcji (local.rules)	24
3.2.2.	Optymalizacja procesu uruchamiania	25
3.3.	Wdrożenie i konfiguracja systemu Suricata	25
3.4.	Implementacja hybrydowego modelu IDS	26
3.4.1.	Algorytm korelacji	27
3.5.	Procedura przeprowadzania eksperymentów	28
3.5.1.	Automatyzacja środowiska pomiarowego	28
3.5.2.	Scenariusze generowania ruchu (Atak)	29
3.5.3.	Przetwarzanie i agregacja wyników	31
4	Wyniki badań eksperymentalnych i analiza wydajności	33
4.1.	Analiza ataku TCP SYN Flood	33
4.1.1.	Analiza obciążenia zasobów sprzętowych	33
4.2.	Analiza ataku UDP Flood	34
4.2.1.	Analiza niestandardowego zachowania Suricaty	34
4.3.	Analiza ataku ICMP Flood	35
4.3.1.	Redukcja szumu informacyjnego	36
4.4.	Analiza ataku HTTP Flood	37
4.4.1.	Koszt analizy DPI (Deep Packet Inspection)	37
4.5.	Analiza czasu detekcji (MTTD)	38
4.6.	Efektywność operacyjna i redukcja szumu (Alert Fatigue)	39
4.7.	Podsumowanie wyników i rekomendacje	40
5	Podsumowanie i wnioski końcowe	41
	Bibliografia	43
	Spis rysunków	45
	Spis tabel	47
A	Wyniki szczegółowe pomiarów	49
B	Kody źródłowe skryptów automatyzujących	52
2.1.	Skrypt sterujący eksperymentem (automated_test.sh)	52
2.2.	Skrypt analizujący logi (analyze_test_extended.sh)	55
2.3.	Moduł korelacji hybrydowej (correlator.py)	56

Wykaz symboli, oznaczeń i skrótów

CPU - Central Processing Unit (procesor)
CSV - Comma-Separated Values
DDoS - Distributed Denial of Service (rozproszona odmowa usługi)
DR - Detection Rate (współczynnik wykrywalności)
FN - False Negative (fałszywie ujemny - niewykryty atak)
FNR - False Negative Rate (wskaźnik błędu braku detekcji)
FP - False Positive (fałszywie dodatni - fałszywy alarm)
FPR - False Positive Rate (wskaźnik fałszywych alarmów)
HIDS - Host-based Intrusion Detection System
HTTP - Hypertext Transfer Protocol
ICMP - Internet Control Message Protocol
IDS - Intrusion Detection System (system wykrywania intruzów)
IIS - Internet Information Services
IoT - Internet of Things
IP - Internet Protocol
IPS - Intrusion Prevention System (system zapobiegania włamaniom)
IT - Information Technology
OISF - Open Information Security Foundation
JSON - JavaScript Object Notation
LAN - Local Area Network
ML - Machine Learning
MTTD - Mean Time To Detect
NAT - Network Address Translation
NIDS - Network-based Intrusion Detection System
RAM - Random Access Memory
SIEM - Security Information and Event Management
TCP - Transmission Control Protocol
TLS - Transport Layer Security
TN - True Negative (prawdziwie ujemny - poprawnie zignorowany ruch)
TP - True Positive (prawdziwie dodatni - poprawnie wykryty atak)
UDP - User Datagram Protocol
VM - Virtual Machine

Wstęp

W erze gwałtownego rozwoju technologii informatycznych, globalnej łączności sieciowej oraz intensywnego wykorzystania usług on-line, zagadnienia bezpieczeństwa infrastruktury IT nabierają szczególnego znaczenia. Jednym z kluczowych wyzwań w obszarze cyberbezpieczeństwa pozostają ataki typu Distributed Denial of Service (DDoS), które poprzez zalewanie zasobów systemu ofiary ogromną liczbą żądań, uniemożliwiają dostęp do usług i skutecznie naruszają atrybut dostępności (ang. *Availability*).

Statystyki branżowe jednoznacznie wskazują na rosnącą skalę tego zagrożenia. W czwartym kwartale 2024 roku firma Cloudflare odnotowała około 6,9 mln ataków DDoS, co stanowiło wzrost o 16% w porównaniu z poprzednim kwartałem oraz o 83% w ujęciu rocznym [1]. Równolegle obserwuje się eskalację wolumenu ataków, czego przykładem są incydenty osiągające szczytowy przepływ rzędu 1,4 Tb/s oraz intensywność przekraczającą 459 mln pakietów na sekundę [2].

W obliczu tak zdefiniowanego zagrożenia, samo wdrożenie systemów wykrywania intruzów (IDS), takich jak Snort czy Suricata, bywa niewystarczające. W domyślnych konfiguracjach systemy te często generują dużą liczbę fałszywych alarmów (ang. *false positives*) lub nie radzą sobie z korelacją zdarzeń rozproszonych w czasie. Prowadzi to do zjawiska „zmęczenia alarmami” (ang. *alert fatigue*), co utrudnia administratorom skuteczną reakcję. Z tego względu konieczne staje się nie tylko wdrożenie, ale przede wszystkim optymalizacja tych narzędzi.

Celem głównym niniejszej pracy jest zaawansowana analiza porównawcza oraz optymalizacja wydajności systemów IDS pod kątem wykrywania i reagowania na ataki DDoS w środowisku korporacyjnym. Realizacja tego celu obejmowała zaprojektowanie i wdrożenie dedykowanego środowiska testowego, składającego się z odseparowanych maszyn wirtualnych pełniących role ofiary, sensora IDS oraz generatora ruchu ofensywnego.

W pracy zaproponowano i przebadano autorskie ulepszenia konfiguracji oraz logiki detekcji. Główne zalety wprowadzonych rozwiązań to:

- **Redukcja szumu informacyjnego:** Dzięki zastosowaniu autorskiego modelu hybrydowego i dostrojeniu reguł, znacząco ograniczono liczbę fałszywych alarmów (FPR).
- **Automatyzacja analizy:** Zaimplementowany mechanizm korelacji alertów pozwala na szybsze identyfikowanie rzeczywistych incydentów, co skraca czas reakcji (MTTD).

- **Większa czytelność:** Wizualizacja wyników umożliwia natychmiastową ocenę skali ataku, co stanowi przewagę nad analizą surowych logów tekstowych.

W ramach badań eksperymentalnych przeanalizowano szerokie spektrum wektorów ataku, obejmujące techniki SYN Flood, UDP Flood, ICMP Flood oraz HTTP Flood o zróżnicowanej intensywności.

Układ pracy został zaprojektowany tak, aby logicznie przeprowadzić czytelnika przez proces badawczy. **Rozdział pierwszy** stanowi wprowadzenie teoretyczne, definiujące istotę systemów IDS, klasyfikację metod detekcji oraz mechanikę ataków wolumenowych. W **rozdziale drugim** szczegółowo opisano metodykę badań, architekturę środowiska testowego oraz scenariusze pomiarowe, uzasadniając dobór konkretnych narzędzi. **Rozdział trzeci** poświęcono aspektom implementacyjnym, prezentując konfigurację silników detekcji oraz proces automatyzacji testów. Kluczowym elementem pracy jest **rozdział czwarty**, zawierający wizualizację i szczegółową analizę uzyskanych wyników wydajnościowych, która weryfikuje skuteczność wprowadzonych optymalizacji. Całość zamyka **rozdział piąty**, w którym zawarto syntezę wniosków końcowych oraz rekomendacje dla administratorów bezpieczeństwa.

Rozdział 1

Wprowadzenie do teorii systemów detekcji intruzów

Zrozumienie mechanizmów działania współczesnych systemów bezpieczeństwa sieciowego jest niezbędne do właściwej oceny ich skuteczności. Niniejszy rozdział wprowadza fundamentalne pojęcia związane z detekcją zagrożeń, omawiając architekturę oraz klasyfikację Systemów Wykrywania Intruzów (IDS). Przedstawiono w nim również charakterystykę ataków typu DDoS, ze szczególnym uwzględnieniem metod ich przeprowadzania, takich jak SYN Flood czy UDP Flood. W końcowej części rozdziału zdefiniowano metryki matematyczne, które posłużą w dalszej części pracy do ilościowej oceny badanych rozwiązań.

1.1. Systemy wykrywania wtargnięć (IDS) - charakterystyka

Systemy wykrywania wtargnięć (Intrusion Detection System, IDS) stanowią jedną z fundamentalnych kategorii mechanizmów ochrony infrastruktury informatycznej. Ich podstawowym zadaniem jest identyfikacja naruszeń bezpieczeństwa (np. skanowanie portów, anomalie ruchu) oraz dostarczanie danych do systemów SIEM (Security Information and Event Management) w celu korelacji zdarzeń.

IDS może pełnić rolę pasywnego detektora (wykrywanie i raportowanie) lub - w architekturze zintegrowanej z systemami zapobiegania (IPS) - także rolę aktywnego elementu blokującego ruch. W praktyce wdrożenia korporacyjne wykorzystują IDS jako część wielowarstwowej strategii zabezpieczeń, integrując go z firewallem, systemami uwierzytelniania oraz narzędziami analitycznymi [3].

W literaturze akcentuje się dwie zasadnicze funkcje IDS:

- Detekcję i klasyfikację zdarzeń bezpieczeństwa.
- Dostarczenie kontekstowych informacji (metadanych) niezbędnych do szybkiej reakcji i późniejszego dochodzenia.

Wiarygodność i użyteczność systemu IDS w praktyce korporacyjnej zależą od jakości reguł/algorytmów w detekcji, zdolności do skalowania w warunkach dużego natężenia ruchu oraz integracji z procesami operacyjnymi (np. playbooki reagowania) [4].

1.2. Klasyfikacja IDS: NIDS vs HIDS

Podstawowe rozróżnienie w systemach wykrywania intruzji przebiega pomiędzy rozwiązaniami działającymi na poziomie sieci (Network-based IDS - NIDS) a rozwiązaniami monitorującymi pojedyncze hosty (Host-based IDS - HIDS).

NIDS analizuje ruch przepływający przez segmenty sieciowe, przechwytyując pakiety i budując strumień (flow) celem analizy protokołów, sekwencji pakietów i sygnatur. Typowe wdrożenie NIDS umieszcza sensor w strategicznym miejscu sieci (np. przed serwerem brzegowym lub za load-balancerem) tak, aby uchwycić ruch przychodzący i wychodzący. NIDS ma zaletę wczesnego wykrywania ataku rozsyłanego na wiele ofiar, ale może być narażony na utratę widoczności w szyfrowanych połączeniach TLS lub przy dużych prędkościach ruchu bez użycia akceleratorów sieciowych [5].

HIDS z kolei instaluje czujniki i agenty bezpośrednio na docelowych systemach (serwery aplikacyjne, bazy danych), monitorując logi systemowe, zmiany plików, wywołania systemowe i zachowanie procesów. Dzięki temu HIDS jest w stanie wychwycić ataki wewnątrz hosta, eskalacje uprawnień oraz działania, które nie manifestują się bezpośrednio na poziomie sieci (np. zmodyfikowane pliki konfiguracyjne). HIDS daje dogłębny kontekst, ale jego wdrożenie i zarządzanie w licznych hostach jest bardziej kosztowne i wymaga centralnego mechanizmu agregacji logów.

W praktyce korporacyjnej często stosuje się hybrydę NIDS + HIDS: NIDS dla szerokiej obserwacji ruchu i szybkiego wykrywania kampanii ataków DDoS, HIDS dla szczegółowej analizy incydentów na poziomie serwera i weryfikacji fałszywych alarmów. W kontekście badań tej pracy skoncentrowano się na podejściu NIDS, ponieważ ataki DDoS manifestują się przede wszystkim poprzez anomalie ruchu sieciowego i obciążenie zasobów sieciowych [6].

1.3. Metody detekcji: sygnaturowa, anomalii, hybrydowa

Detekcja sygnaturowa (signature-based) opiera się na poszukiwaniu w ruchu sieciowym znanych wzorców zachowań, ciągów bajtów lub sekwencji pakietów skorelowanych z wcześniej zidentyfikowanymi atakami. Systemy oparte na sygnaturach (np. Snort) charakteryzują się wysoką precyzją w rozpoznawaniu znanych zagrożeń i niskim FPR przy dobrze sformułowanych regułach, jednak są słabsze wobec nowych, zmodyfikowanych lub szyfrowanych ataków [7]. W praktyce reguły sygnaturowe

wymagają stałej aktualizacji oraz mechanizmów tłumienia (`threshold/detection_filter`), aby ograniczyć zalewanie alertami w warunkach masowych ataków typu flood.

Detekcja anomalii (*anomaly-based*) polega na zbudowaniu modelu normalnego zachowania (statystycznego lub uczonego przez algorytmy ML) i sygnalizowaniu znaczących odchyleń. Modele mogą dotyczyć cech ruchu (np. liczba połączeń na sekundę, rozkład rozmiarów pakietów, entropia nagłówek) lub zachowań aplikacyjnych. Metody te wykazują przewagę w wykrywaniu nowych, wcześniej nieznanymi ataków, jednak ich efektywność zależy od jakości i reprezentatywności danych treningowych [8]. W szczególności model „baseline” musi uwzględniać sezonowość i profile ruchu w środowisku korporacyjnym, aby ograniczyć fałszywe alarmy.

Podjęcie hybrydowe łączy reguły sygnaturowe z modelami anomalii oraz często wykorzystuje mechanizmy korelacji alertów i ensemble learning w celu poprawy zarówno detekcji, jak i redukcji FPR. Hybrydowe systemy mogą wykonywać wstępną filtrację poprzez sygnatury, a następnie poddawać podejrzanym zdarzeniom głębszej analizie anomalii (ang. *two-stage detection*), co jest efektywne w środowiskach dużego ruchu. Architektury hybrydowe, łączące szybkość sygnatur z analizą anomalii, wykazują wyższą skuteczność w detekcji złożonych ataków DDoS, co potwierdzają badania Patchy i Parka [8].

1.4. Ataki DDoS - istota, typy i mechanizmy

Atak typu Distributed Denial of Service polega na masowym, skoordynowanym generowaniu ruchu z wielu źródeł w celu wyczerpania zasobów obliczeniowych, pamięci lub przepustowości łącza ofiary. Według klasycznej topologii Mirkovica i Reihera, ataki DDoS mają na celu „wyczerpanie dostępnych zasobów ofiary poprzez skierowanie do niej ogromnej liczby żądań z wielu rozproszonych węzłów” [9].

W literaturze wyróżnia się trzy podstawowe kategorie ataków DDoS:

- **Ataki wolumenowe (volume-based)** - ich celem jest przeciążenie łącza sieciowego lub saturacja pasma. Do tej grupy zalicza się m.in. UDP Flood, powodujący zalanie hosta pakietami UDP, co często generuje odpowiedzi ICMP Destination Unreachable, oraz ICMP Flood, polegający na wysyłaniu dużej liczby pakietów ICMP Echo Request [10].
- **Ataki protokolarne (protocol-based)** - wykorzystują słabości w implementacji protokołów komunikacyjnych. Najpopularniejszy jest SYN Flood, który wymusza na serwerze alokację zasobów w tablicy połączeń, nie doprowadzając do ich finalizacji [10].

- **Ataki warstwy aplikacyjnej (application-layer)** - ukierunkowane na obciążenie zasobów aplikacyjnych, często przy użyciu ruchu wyglądającego na normalny. Typowym przykładem jest HTTP GET/POST Flood, polegający na generowaniu licznych zapytań HTTP o wysokim koszcie obsługi [11].

Współczesne kampanie DDoS są często wielowektorowe, co oznacza, że łączą jednocześnie różne techniki np. SYN Flood + UDP Flood + HTTP Flood. Zwiększa to trudność detekcji oraz wymaga stosowania mechanizmów korelacji alertów i elastycznych metod filtracji.

Istotnym zjawiskiem ostatnich lat jest gwałtowny wzrost skali ataków. Zargar i współautorzy wskazują, że największe kampanie osiągają wartości rzędu terabitów na sekundę, a liczba pakietów na sekundę rośnie do setek milionów. Wang i inni podkreślają, że kluczową rolę w eskalacji skali odgrywają botnety oparte na urządzeniach IoT, które dzięki dużej liczbie słabo zabezpieczonych urządzeń są w stanie generować złożony i trudny do filtrowania ruch [12].

Podsumowując, najczęściej analizowane w literaturze typy ataków DDoS to:

- SYN Flood - przeciążenie tablic połączeń TCP,
- UDP Flood - nadmierne obciążenie łącza i generowanie ruchu zwrotnego,
- ICMP Flood - wysycenie zasobów hosta pakietami ICMP,
- HTTP Flood - przeciążenie warstwy aplikacyjnej.

1.5. Metryki skuteczności IDS

Ocena skuteczności systemu wykrywania wtargnięć (IDS) nie może opierać się wyłącznie na subiektywnej obserwacji, lecz wymaga precyzyjnego zdefiniowania miar jakościowych i ilościowych. W badaniach nad systemami detekcji ataków DDoS standardem jest stosowanie metryk opartych na macierzy pomyłek (ang. *Confusion Matrix*), która pozwala na klasyfikację decyzji systemu.

Do fundamentalnych wielkości opisujących stan detekcji należą:

- **True Positive (TP)** - poprawnie wykryte zdarzenie ataku (atak rozpoznany jako atak).
- **False Positive (FP)** - błędny alarm; normalny ruch zaklasyfikowany jako atak.
- **True Negative (TN)** - poprawne zignorowanie normalnego ruchu.
- **False Negative (FN)** - niewykrycie ataku; atak zaklasyfikowany jako ruch normalny.

Na podstawie powyższych składowych wyznacza się wskaźniki pochodne, które pozwalają na liczbową ocenę jakości detekcji.

Pierwszym kluczowym wskaźnikiem jest **Współczynnik Wykrywalności** (ang. *Detection Rate* - DR), nazywany również czułością (Recall). Określa on, jaką część wszystkich rzeczywistych ataków system był w stanie poprawnie zidentyfikować. Wzór definiujący tę metrykę przyjmuje postać:

$$DR = \frac{TP}{TP + FN} \quad (1.1)$$

Wysoka wartość DR jest pożądana, jednak analizowana w izolacji może być myląca, jeśli system generuje przy tym ogromną liczbę fałszywych alarmów.

Dlatego równie istotną metryką jest **Współczynnik Fałszywych Alarmów** (ang. *False Positive Rate* - FPR). Opisuje on prawdopodobieństwo, że system błędnie oflaguje bezpieczny ruch jako zagrożenie. Matematycznie wyraża się go jako stosunek fałszywych alarmów do wszystkich zdarzeń, które w rzeczywistości były bezpieczne:

$$FPR = \frac{FP}{FP + TN} \quad (1.2)$$

W kontekście operacyjnym, wysoki wskaźnik FPR jest krytycznym problemem. Generuje on "szum informacyjny", który podnosi koszty obsługi systemu i prowadzi do desensytyzacji personelu bezpieczeństwa. Celem optymalizacji systemów IDS jest zatem maksymalizacja DR przy jednoczesnej minimalizacji FPR [13].

Uzupełnieniem tych miar jest **Mean Time To Detect (MTTD)**, czyli średni czas od wystąpienia anomalii do wygenerowania alertu, oraz wskaźniki wydajnościowe, takie jak zużycie zasobów (CPU/RAM) przez sensor IDS.

1.6. Wyzwania optymalizacji IDS w środowiskach korporacyjnych

Wdrożenie systemu IDS w środowisku laboratoryjnym różni się znacząco od jego utrzymania w dynamicznych sieciach produkcyjnych o wysokiej przepustowości (ang. *High-Speed Networks*). Administratorzy i inżynierowie bezpieczeństwa stają przed wyzwaniami, które można podzielić na trzy główne obszary: wydajność przetwarzania, redukcję fałszywych alarmów oraz zarządzanie regułami.

1.6.1. Wydajność i przetwarzanie w czasie rzeczywistym

W sieciach o przepustowości rzędu gigabitów na sekundę, IDS musi analizować każdy pakiet w czasie rzeczywistym. Niedostateczna wydajność sensora prowadzi do dwóch negatywnych zjawisk:

- **Opóźnienia (Latency):** System musi podejmować decyzje w ułamku sekundy. Zbyt długi czas analizy opóźnia reakcję, co w przypadku systemów IPS (blokujących ruch) może degradować jakość usług sieciowych dla użytkowników.
- **Gubienie pakietów (Packet dropping):** Gdy obciążenie przekracza możliwości obliczeniowe sensora (wąskie gardło), system zaczyna odrzucać pakiety bez ich analizy. Stanowi to poważną lukę bezpieczeństwa, gdyż atakujący może wykorzystać moment saturacji łącza do przemycenia złośliwego ruchu.

1.6.2. Problem nadmiarowości alarmów

Wspomniany wcześniej wskaźnik FPR przekłada się bezpośrednio na efektywność operacyjną zespołów SOC (Security Operations Center). Nadmiar fałszywych alarmów powoduje dwa rodzaje ryzyk:

- **Ekonomiczne:** Każdy alarm musi zostać zweryfikowany, co angażuje czas analityków. Wysoki FPR oznacza marnowanie zasobów ludzkich na analizę zdarzeń nieistotnych.
- **Psychologiczne (Alert Fatigue):** Chroniczny nadmiar alarmów prowadzi do spadku czujności. Istnieje ryzyko, że w gąszczu fałszywych powiadomień, rzeczywisty i krytyczny incydent (True Positive) zostanie potraktowany rutynowo i zignorowany.

1.6.3. Złożoność zarządzania regułami

W systemach sygnaturowych, takich jak Snort czy Suricata, baza reguł może liczyć dziesiątki tysięcy wpisów. Aktywowanie wszystkich dostępnych reguł "na wszelki wypadek" drastycznie obniża wydajność silnika, zmuszając go do porównywania każdego pakietu z ogromną bazą wzorców.

Współczesne podejście do optymalizacji, które będzie również przedmiotem badań w niniejszej pracy, polega na selektywnym doborze reguł (tuning) oraz automatyzacji tego procesu. Celem jest stworzenie konfiguracji dopasowanej do profilu ruchu konkretnej organizacji, co pozwala na osiągnięcie równowagi między bezpieczeństwem a wydajnością [14].

Rozdział 2

Opis eksperymentów i metody analizy wyników

Niniejszy rozdział przedstawia szczegółowy opis procesu badawczego, mającego na celu weryfikację skuteczności systemów detekcji intruzów. Omówiono w nim budowę izolowanego środowiska testowego, konfigurację wykorzystanych narzędzi (Snort, Suricata) oraz procedurę generowania ataków DDoS. Przedstawiono również autorski mechanizm korelacji zdarzeń oraz sposób automatyzacji pomiarów, co zapewnia powtarzalność przeprowadzonych eksperymentów.

2.1. Cel, zakres i założenia badań

Głównym celem badań jest empiryczna ocena skuteczności systemów IDS w detekcji ataków wolumetrycznych i aplikacyjnych w środowisku korporacyjnym. Realizacja tego celu polegała na zaprojektowaniu laboratorium wirtualnego odwzorowującego fragment sieci firmowej, implementacji dwóch wiodących rozwiązań klasy IDS, przeprowadzeniu serii kontrolowanych ataków oraz analizie wyników pod kątem wskaźników jakości detekcji.

Przyjęto następujące założenia projektowe:

- **Izolacja środowiska:** Praca wszystkich maszyn odbywa się w zamkniętym segmencie sieci LAN, co eliminuje wpływ losowego ruchu z Internetu oraz zapewnia bezpieczeństwo infrastruktury macierzystej.
- **Porównywalność:** Równoległe uruchomienie procesów Snort i Suricata na tej samej maszynie wirtualnej (z identycznym dostępem do zasobów) umożliwia bezpośrednie porównywanie ich wydajności.
- **Stopniowanie zagrożenia:** Zastosowanie trzystopniowej skali intensywności ataków (niska, średnia, wysoka) pozwala zbadać punkt nasycenia sensorów.
- **Korelacja hybrydowa:** Zastosowanie autorskiego skryptu integrującego wyniki z obu systemów w celu weryfikacji hipotezy o wyższej skuteczności modelu hybrydowego.

Badania miały charakter eksperymentalny – w każdej iteracji zmieniano jeden czynnik (wektor lub intensywność ataku), utrzymując pozostałe parametry stałe (*ceteris paribus*).

2.2. Opis środowiska testowego

2.2.1. Struktura i oprogramowanie

Środowisko badawcze zrealizowano w oparciu o platformę wirtualizacji VMware Workstation 17 Pro. Wybór ten podyktowany był koniecznością bezpiecznej separacji złośliwego ruchu oraz łatwością zarządzania migawkami (snapshots) systemów. Utworzono trzy wirtualne maszyny połączone w dedykowany segment sieciowy LAN (Tab. 2.1).

Tab. 2.1. Struktura i oprogramowanie środowiska testowego

Nazwa maszyny	System operacyjny	Rola	Konfiguracja sprzętowa	Uwagi
Windows 7 (ofiara)	Windows 7 SP1 x64	Serwer docelowy (ofiara)	3GB RAM, 2 rdzenie CPU, 90GB HDD	System pełni rolę serwera atakowanego, zainstalowano IIS.
IDS	Ubuntu 22.04 LTS	Sensor IDS	6GB RAM, 2 rdzenie CPU, 124GB HDD, 2 interfejsy sieciowe	Na tej maszynie zainstalowano Snort 2.9 oraz Suricata 8.0.1. Interfejs ens37 obsługuje analizę ruchu między ofiarą a atakującym.
Kali Linux	Kali 2025.3	Generator ataków DDoS	3GB RAM, 4 rdzenie CPU, 80GB HDD	Służy do symulacji ataków przy użyciu narzędzi hping3 oraz Apache Benchmark.

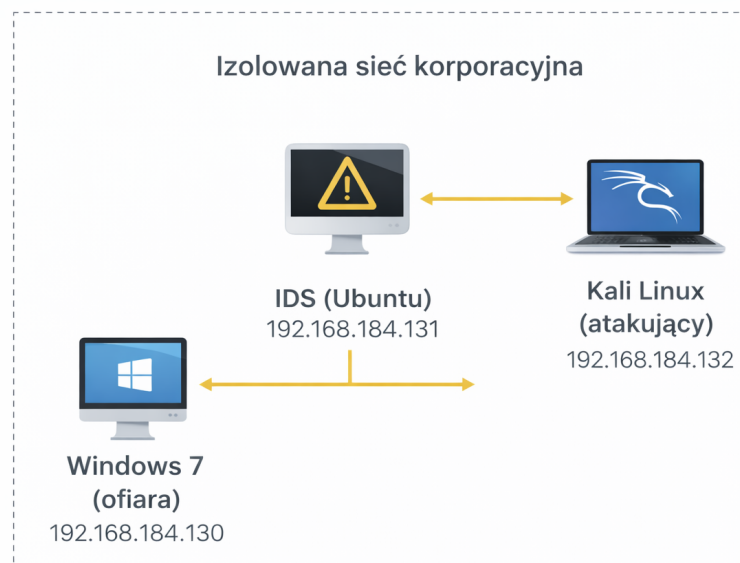
2.2.2. Organizacja sieci i topologia

Maszyny zostały połączone w wewnętrzny segment sieciowy (LAN Segment) o nazwie „ids-lab”. Gwarantuje to, że generowany ruch flood nie wydostanie się poza środowisko testowe. Sensor IDS wyposażono w dwa interfejsy sieciowe:

- **Interfejs zarządzający (NAT):** Umożliwia zdalny dostęp i aktualizację reguł.
- **Interfejs nasłuchujący (LAN Segment):** Pracuje w trybie promiskuitywnym (promiscuous mode), przechwytyjąc cały ruch na linii Atakujący – Ofiara.

Adresacja sieciowa poszczególnych maszyn:

- Windows 7 (ofiara) – 192.168.184.130
- IDS (Ubuntu) – 192.168.184.131
- Kali Linux (atakujący) – 192.168.184.132



Rys. 2.1. Schemat topologii środowiska testowego w VMware. Źródło: Opracowanie własne

Logiczne umiejscowienie sensora IDS (Rys. 2.1) odpowiada typowej architekturze korporacyjnej, gdzie system detekcji analizuje ruch wchodzący do strefy DMZ przed serwerami aplikacyjnymi.

2.3. Konfiguracja systemów IDS

2.3.1. Snort

Snort skonfigurowano jako pasywny sensor sieciowy (NIDS). Główny plik konfiguracyjny `/etc/snort/snort.conf` został zmodyfikowany w celu zdefiniowania chronionej podsieci:

```
1 var HOME_NET 192.168.184.0/24
2 var EXTERNAL_NET any
3 include /etc/snort/rules/local.rules
```

W pliku `local.rules` zaimplementowano dedykowane reguły detekcji dla badanych wektorów ataków (SYN, UDP, ICMP, HTTP). Przykładowa reguła wykrywająca anomalię TCP SYN:

```
1 alert tcp any any -> 192.168.184.130 80 (flags:S; msg:"Possible TCP
  SYN Flood Attack"; flow:stateless; detection_filter:track by_dst,
  count 100, seconds 10; sid:100001; rev:1;)
```

Snort uruchamiany był w trybie demona z wyjściem w formacie „fast” (uproszczony tekst), co ułatwia późniejsze parsowanie:

```
1 sudo snort -A fast -q -c /etc/snort/snort.conf -i ens37 -D
```

2.3.2. Suricata

Drugi z badanych systemów, Suricata, został skonfigurowany do pracy z wielowątkowym przetwarzaniem pakietów. W pliku `suricata.yaml` aktywowano wyjście w standardzie EVE JSON (`eve-log`), co pozwala na zapis bogatych metadanych o każdym zdarzeniu.

Uruchomienie sensora następowało poleceniem:

```
1 sudo suricata -c /etc/suricata/suricata.yaml -i ens37 -D
```

Wyniki detekcji zapisywane były w pliku `/var/log/suricata/eve.json`, a statystyki obciążenia silnika w `stats.log`.

2.3.3. Tryb hybrydowy (Korelacja)

W celu eliminacji słabości pojedynczych silników detekcji (np. wysoki FPR w systemach sygnaturowych), opracowano podejście hybrydowe. Polega ono na programowej korelacji alertów pochodzących niezależnie od Snorta i Suricaty.

Logika korelacji została zaimplementowana w języku Python. Skrypt parsuje logi z obu systemów i łączy je w jeden incydent, jeżeli spełnione są dwa warunki:

1. Zgodność adresu IP ofiary i atakującego.
2. Różnica czasu wystąpienia alertów (Δt) mniejsza niż 2 sekundy.

Takie podejście (głosowanie większościowe) ma na celu odfiltrowanie przypadkowych fałszywych alarmów, które rzadko są generowane przez dwa różne silniki w tej samej milisekundzie.

2.4. Scenariusze i procedura testowa

2.4.1. Rodzaje ataków

W celu uzyskania kompleksowej oceny wydajności, przeprowadzono cztery scenariusze ataków, pokrywające warstwy L3, L4 oraz L7 modelu ISO/OSI. Do generowania ruchu wykorzystano narzędzia `hping3` (dla warstw niższych) oraz `Apache Benchmark` (dla warstwy aplikacji).

- **SYN Flood:** Wykorzystuje wadę protokołu TCP (handshake), zalewając serwer pakietami z flagą SYN w celu wyczerpania tablicy połączeń.
- **UDP Flood:** Atak wolumenowy polegający na wysyłaniu pakietów UDP na losowe porty, zmuszając ofiarę do generowania odpowiedzi ICMP Destination Unreachable.
- **ICMP Flood:** Masowe wysyłanie komunikatów Echo Request (Ping), prowadzące do saturacji pasma.
- **HTTP Flood:** Atak warstwy aplikacyjnej generujący poprawne składniowo zapytania GET/POST, mające na celu przeciążenie serwera WWW.

Każdy test trwał 30 sekund, po czym następowała 60-sekundowa przerwa techniczna (Cool-down) na opróżnienie buforów systemowych.

2.4.2. Poziomy intensywności

Dla każdego wektora ataku zdefiniowano trzy poziomy natężenia ruchu, sterowane parametrem interwału wysyłania pakietów:

- **Low (Niska):** ok. 1 000 – 5 000 pps. Symulacja wczesnej fazy ataku lub działania pojedynczego bota.
- **Medium (Średnia):** ok. 10 000 – 20 000 pps. Znaczne obciążenie, typowe dla mniejszych botnetów.
- **High (Wysoka):** Maksymalna przepustowość łącza (tryb `-flood`). Symulacja zmasowanego ataku DDoS mającego na celu całkowite odcięcie usługi.

Przykładowe komendy użyte w eksperymentach dla ataku SYN Flood:

```
1 # Low (interwał 1000us)
2 sudo hping3 -S -p 80 -i u1000 192.168.184.130
3 # Medium (interwał 100us)
4 sudo hping3 -S -p 80 -i u100 192.168.184.130
5 # High (flood - brak interwału)
6 sudo hping3 -S -p 80 --flood 192.168.184.130
```

Listing 2.1. Komendy sterujące intensywnością ataku w `hping3`

2.4.3. Automatyzacja procesu

Aby zapewnić obiektywizm wyników, procedura testowa została w pełni zautomatyzowana. Skrypt sterujący `automated_test.sh` realizował sekwencję: czyszczenie logów → restart usług IDS → uruchomienie monitoringu zasobów → oczekiwanie na atak → archiwizacja wyników. Każdy pomiar powtarzano trzykrotnie, a wynik końcowy uśredniano.

2.5. Zbieranie i analiza danych

Dane wyjściowe gromadzone były w formatach natywnych dla każdego systemu (tekstowy dla Snort, JSON dla Suricata). Do ich przetwarzania wykorzystano zestaw skryptów Bash wspieranych narzędziami takimi jak `grep`, `awk` oraz `jq`.

Poniższy fragment kodu prezentuje metodę precyzyjnej ekstrakcji liczby alertów z seryjnego pliku JSON generowanego przez Suricatę:

```
1 if [ -f "$SURICATA_LOG" ]; then
2     # Filtrowanie tylko zdarzeń typu 'alert' i ich zliczanie
3     SURICATA_ALERTS=$(jq -r 'select(.event_type=="alert") | .
4     event_type' \
5     "$SURICATA_LOG" | wc -l)
6 else
7     SURICATA_ALERTS=0
8 fi
```

Listing 2.2. Fragment skryptu parsujący logi Suricaty (`jq`)

Dane surowe po przetworzeniu trafiały do zbiorczego pliku `results_extended.csv`, stanowiącego bazę do analizy w Rozdziale 4.

2.6. Metodyka oceny skuteczności

Ilościowa ocena badanych rozwiązań została przeprowadzona w oparciu o metryki zdefiniowane teoretycznie w Rozdziale 1 (sekcja 1.5). W procesie analizy logów automatycznie wyliczano następujące wskaźniki:

- **Detection Rate (DR):** Obliczany jako stosunek liczby wygenerowanych alertów do liczby wysłanych pakietów ofensywnych (dla ataków wolumetrycznych) lub liczby żądań (dla HTTP).
- **False Positive Rate (FPR):** Mierzony podczas testów ruchu referencyjnego (bez ataku), określający liczbę błędnych zgłoszeń w jednostce czasu.

Uzupełnieniem analizy skuteczności detekcji jest pomiar kosztu obliczeniowego. Monitorowano zużycie procesora (CPU Usage %) oraz pamięci operacyjnej (RAM Usage MB) przez procesy `snort` i `Suricata-Main` w trakcie trwania ataku. Pozwala to na ocenę, jak badane systemy radzą sobie ze skalowalnością w warunkach stresowych.

2.7. Walidacja i ograniczenia metodyki

W celu minimalizacji błędów pomiarowych zastosowano trzykrotne powtórzenie każdego scenariusza testowego. Weryfikowano również spójność znaczników czasowych (timestamps) między maszyną atakującą a sensorem, aby wykluczyć opóźnienia wynikające z wirtualizacji.

Należy jednak wskazać na pewne ograniczenia przyjętej metodyki:

- **Syntetyczność ruchu:** Środowisko laboratoryjne pozbawione jest losowego szumu (background traffic), który występuje w sieciach produkcyjnych, co może wpływać na nieco niższy poziom False Positives niż w rzeczywistości.
- **Narzut wirtualizacji:** Wydajność interfejsów sieciowych maszyn wirtualnych jest niższa niż sprzętowych kart sieciowych, co może stanowić wąskie gardło przy testach typu High Flood.

Mimo tych ograniczeń, środowisko zapewnia stabilne i, co kluczowe, powtarzalne warunki do analizy porównawczej algorytmów detekcji.

Rozdział 3

Implementacja środowiska badawczego i przebieg eksperymentów

Rozdział ten poświęcono praktycznym aspektom przygotowania infrastruktury badawczej oraz realizacji scenariuszy testowych. Szczegółowo omówiono proces instalacji i konfiguracji sensorów IDS (Snort i Suricata), ze szczególnym uwzględnieniem optymalizacji reguł detekcji pod kątem ataków wolumetrycznych. W dalszej części przedstawiono implementację autorskiego algorytmu korelacji zdarzeń (model hybrydowy) oraz metodykę automatyzacji testów przy użyciu skryptów powłoki Bash i języka Python.

3.1. Konfiguracja maszyn wirtualnych i infrastruktury testowej

Zgodnie z projektem przedstawionym w Rozdziale 2, środowisko eksperymentalne oparto na virtualizacji w programie VMware Workstation 17 Pro. Kluczowym elementem konfiguracji było utworzenie odseparowanego segmentu sieciowego (LAN Segment o nazwie *ids-lab*), co pozwoliło na symulację rzeczywistej architektury sieciowej typu „Inside/Outside” bez ryzyka wycieku szkodliwego ruchu do sieci macierzystej.

Wszystkie maszyny wirtualne zostały skonfigurowane zgodnie ze specyfikacją sprzętową ujętą w Tabeli 2.1, a ich role (Ofiara, Atakujący, Sensor) zostały ściśle zdefiniowane systemowo. Niniejszy rozdział koncentruje się na warstwie programowej – wdrożeniu silników detekcji oraz narzędzi analitycznych.

3.2. Wdrożenie i konfiguracja systemu Snort

Jako pierwszy element systemu bezpieczeństwa zaimplementowano Snort w wersji 2.9. Proces instalacji na systemie Ubuntu 22.04 objął pakiety bazowe, biblioteki DAQ (Data Acquisition) oraz zbiory reguł społeczności.

Kluczowym etapem wdrożenia była edycja głównego pliku konfiguracyjnego `snort.conf`. W celu poprawnego rozpoznawania kierunków ruchu, zdefiniowano zmienną `HOME_NET` wskazującą na chronioną podsieć laboratoryjną.

```
#
ipvar HOME_NET 192.168.184.0/24

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any
```

Rys. 3.1. Definicja zmiennej HOME_NET w pliku konfiguracyjnym. Źródło: Opracowanie własne

Następnie aktywowano obsługę lokalnych reguł użytkownika, odkomentowując ścieżkę do pliku `local.rules` (Rys. 3.2). Pozwoliło to na dodanie autorskich sygnatur bez ingerencji w oficjalne bazy reguł.

```
# site specific rules
include $RULE_PATH/local.rules
```

Rys. 3.2. Aktywacja pliku `local.rules` w konfiguracji Snorta. Źródło: Opracowanie własne

3.2.1. Implementacja reguł detekcji (`local.rules`)

Aby skutecznie wykrywać ataki typu flood, standardowe reguły sygnaturowe są niewystarczające, gdyż pojedynczy pakiet SYN czy UDP jest technicznie poprawny. Dlatego w pliku `/etc/snort/rules/local.rules` zaimplementowano reguły wykorzystujące preprocesor `detection_filter` (progowanie).

Mechanizm ten pozwala na zliczanie wystąpień danego zdarzenia w określonym oknie czasowym. Alarm generowany jest dopiero po przekroczeniu zdefiniowanego progu (`threshold`). Poniższy listing prezentuje finalną konfigurację reguł użytych w badaniu:

```
1 # 1. TCP SYN FLOOD
2 alert tcp any any -> 192.168.184.130 80 (flags:S; msg:"SYN Flood
   Attack"; detection_filter:track by_src, count 100, seconds 1; sid
   :1100001; rev:1;)
3
4 # 2. UDP FLOOD
5 alert udp any any -> 192.168.184.130 any (msg:"UDP Flood Attack";
   detection_filter:track by_src, count 20, seconds 1; sid:1100002;
   rev:1;)
6
7 # 3. ICMP FLOOD
8 alert icmp any any -> any any (msg:"ICMP Flood Attack";
   detection_filter:track by_src, count 20, seconds 1; sid:1100003;
   rev:3;)
9
```



```

10 # 4. HTTP GET FLOOD
11 alert tcp any any -> 192.168.184.130 80 (msg:"HTTP Flood Attack"; flow
    :to_server,established; content:"GET /"; detection_filter:track
    by_src, count 50, seconds 5; sid:1100004; rev:1;)

```

Listing 3.1. Zastosowane reguły Snorta z mechanizmem progowania

3.2.2. Optymalizacja procesu uruchamiania

W warunkach laboratoryjnych, gdzie generowany jest ruch o wysokim natężeniu (High Flood), standardowe logowanie do pełnych plików tekstowych może stać się wąskim gardłem (I/O bottleneck). W celu uniknięcia utraty pakietów, Snort uruchamiany był w trybie `fast alert`, który ogranicza zapis logów do niezbędnego minimum (timestamp, IP, port, komunikat).

Komenda uruchamiająca została zaszyta w skrypcie automatyzującym:

```

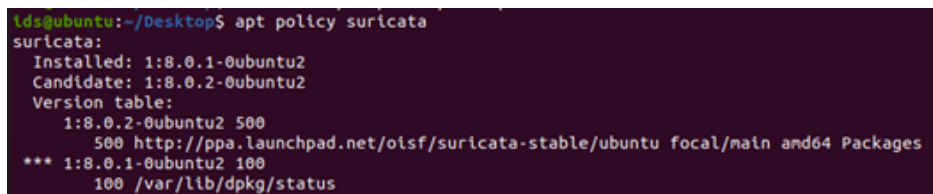
1 sudo snort -A fast -c /etc/snort/snort.conf -i ens37

```

Listing 3.2. Polecenie uruchomienia procesu Snort (tryb fast)

3.3. Wdrożenie i konfiguracja systemu Suricata

Jako drugie rozwiązanie, służące do analizy porównawczej, zainstalowano system Suricata. Wykorzystano oficjalne repozytorium OISF (Open Information Security Foundation), co zagwarantowało dostęp do najnowszej stabilnej wersji silnika.



```

lds@ubuntu:~/Desktop$ apt policy suricata
suricata:
  Installed: 1:8.0.1-0ubuntu2
  Candidate: 1:8.0.2-0ubuntu2
  Version table:
   1:8.0.2-0ubuntu2 500
        500 http://ppa.launchpad.net/oisf/suricata-stable/ubuntu focal/main amd64 Packages
   *** 1:8.0.1-0ubuntu2 100
        100 /var/lib/dpkg/status

```

Rys. 3.3. Weryfikacja instalacji pakietu Suricata. Źródło: Opracowanie własne

Kluczowym aspektem konfiguracji, różniącym Suricatę od Snorta, jest wielowątkowość. W pliku `suricata.yaml` skonfigurowano tryb przechwytywania `af-packet`, który jest natywnym i wysokowydajnym mechanizmem w systemach Linux, pozwalającym na równoległe przetwarzanie potoków ruchu.

```
# Linux high speed capture support
af-packet:
  - interface: ens37
```

Rys. 3.4. Konfiguracja trybu af-packet dla interfejsu nasłuchującego. Źródło: Opracowanie własne

Dla potrzeb późniejszej analizy danych (Data Science), kluczowe było włączenie logowania w formacie strukturalnym. Aktywowano wyjście `eve-log` (Extensible Event Format) oparte na JSON, co umożliwia łatwe parsowanie logów przez zewnętrzne skrypty Python.

```
outputs:
  # a line based alerts log similar to Snort's fast.log
  - fast:
    enabled: yes
    filename: fast.log
    append: yes
    #filetype: regular # 'regular', 'unix_stream' or 'unix_dgram'

  # Extensible Event Format (nicknamed EVE) event log in JSON format
  - eve-log:
    enabled: yes
    filetype: regular #regular|syslog|unix_dgram|unix_stream|redis
    filename: eve.json
```

Rys. 3.5. Aktywacja logowania EVE JSON. Źródło: Opracowanie własne

Dodatkowo, w celu weryfikacji wpływu ataków na sam sensor, włączono moduł `stats`, raportujący cyklicznie zużycie zasobów i liczbę zgubionych pakietów (kernel drops).

```
# Global stats configuration
stats:
  enabled: yes
```

Rys. 3.6. Konfiguracja modułu statystyk wydajności. Źródło: Opracowanie własne

3.4. Implementacja hybrydowego modelu IDS

Istotnym wkładem własnym pracy jest implementacja modelu hybrydowego, integrującego wyniki z obu silników. Zrealizowano go w formie zewnętrznego skryptu korelacyjnego (`correlator.py`), działającego jako warstwa post-processingu.

Celem tego modułu jest weryfikacja hipotezy, że zdarzenie potwierdzone przez dwa niezależne algorytmy detekcji w tym samym oknie czasowym cechuje się wyższą wiarygodnością niż pojedynczy alert.

3.4.1. Algorytm korelacji

Logika działania skryptu opiera się na następujących krokach:

1. **Ingestia danych:** Wczytanie plików logów z obu systemów i ich normalizacja do wspólnego formatu obiektu czasu (datetime).
2. **Filtracja celu:** Odrzucenie alertów nie dotyczących serwera ofiary (192.168.184.130).
3. **Korelacja czasowa:** Wyszukiwanie par zdarzeń (Snort + Suricata), dla których różnica czasu wystąpienia (Δt) nie przekracza zdefiniowanego okna korelacji.

Przyjęto okno korelacji `CORRELATION_WINDOW = 2s`, co uwzględnia różnice w czasie przetwarzania pakietów przez oba silniki. Poniższy fragment kodu prezentuje rdzeń algorytmu:

```
1 # Pętla korelacyjna
2 for u in u_alerts:
3     if u['timestamp'] in used_suricata_ts:
4         continue
5
6     for s in s_alerts:
7         # Obliczanie delty czasowej między alertami z różnych systemów
8         delta = abs((s["timestamp"] - u["timestamp"]).total_seconds())
9
10        if delta <= CORRELATION_WINDOW:
11            # Zidentyfikowano skorelowany incydent
12            incident_count += 1
13            used_suricata_ts.add(u['timestamp'])
14            print(f"HIGH CONFIDENCE INCIDENT: {u['timestamp']}
15 confirmed")
16            break
```

Listing 3.3. Fragment algorytmu korelacji w Pythonie

Zastosowanie takiej warstwy weryfikacyjnej ma na celu redukcję zjawiska *alert fatigue*, poprzez prezentowanie administratorowi jedynie potwierdzonych, wysokoprawdopodobnych incydentów.

3.5. Procedura przeprowadzania eksperymentów

Aby zapewnić rzetelność wyników badawczych, opracowano ustandaryzowaną procedurę testową. Proces ten został w pełni zautomatyzowany przy użyciu skryptu powłoki Bash (`automated_test.sh`), który zarządza cyklem życia każdego eksperymentu.

3.5.1. Automatyzacja środowiska pomiarowego

Skrypt sterujący odpowiada za przygotowanie "czystego" stanu maszyny przed każdą próbą. Jego działanie obejmuje:

- Usunięcie logów z poprzednich testów (zapobieganie zanieczyszczeniu danych).
- Restart usług Snort i Suricata w celu opróżnienia buforów pamięci.
- Utworzenie dedykowanej struktury katalogów dla aktualnego scenariusza (np. `/ddos-tests/syn/low`).
- Uruchomienie w tle procesów monitorujących zasoby (`top`, `vmstat`).

Uruchomienie procedury odbywa się z poziomu terminala sensora IDS:

```
1 sudo ./automated_test.sh
```

Listing 3.4. Inicjacja procedury testowej

```

ids@ubuntu:~/Desktop$ sudo ./automated_test.sh
Podaj typ ataku (syn / udp / icmp / http):
http
Podaj intensywnosc (low / medium / high):
low
[INFO] Czyszczenie starych wynikow w /root/ddos-tests/http/low...
Logi beda zapisane w: /root/ddos-tests/http/low

[INFO] Czyszczenie logow systemowych...
[OK] Logi systemowe wyczyszczone.

[INFO] Uruchamiam Snorta i Suricate na ens37...
[OK] Snort PID: 2912
[OK] Suricata PID: 2913

[INFO] Startuje top i vmstat...
[OK] top PID: 2935
[OK] vmstat PID: 2936

=====
TERAZ WYKONAJ ATAK NA KALI
(np. hping3 --flood -S -p 80 192.168.184.130)
=====

Nacisnij CTRL+C tutaj po zakonczonym ataku, aby zakonczyc test...
^C

[INFO] Zatrzymuje procesy...
[OK] Wszystkie procesy zatrzymane.

[INFO] Kopiowanie logow IDS do folderu...
[OK] Logi skopiowane do /root/ddos-tests/http/low

=====
TEST UKONCZONY
Uruchom teraz: sudo ./analize_test_extended.sh
=====

```

Rys. 3.7. Interfejs skryptu automatyzującego podczas oczekiwania na atak. Źródło: Opracowanie własne

Skrypt działa w trybie interaktywnym, oczekując na wykonanie ataku przez użytkownika, a następnie – po otrzymaniu sygnału przerwania (SIGINT) – automatycznie zamyka wszystkie procesy i archiwizuje logi w odpowiednich folderach.

3.5.2. Scenariusze generowania ruchu (Atak)

Ruch ofensywny generowano z maszyny Kali Linux, realizując cztery scenariusze ataków (SYN, UDP, ICMP, HTTP) w trzech poziomach natężenia. Do ataków warstwy sieciowej (L3/L4) wykorzystano narzędzie `hping3`, natomiast do warstwy aplikacji (L7) – `Apache Benchmark`.

Przykłady komend sterujących dla poszczególnych wektorów ataku przedstawiono poniżej.

1. SYN Flood (Low)

Atak z wykorzystaniem pakietów SYN wysyłanych z interwałem 1000 mikrosekund:

```
1 sudo hping3 -S -p 80 -i u1000 192.168.184.130
```

W powyższym poleceniu flaga **-S** wymusza ustawienie bitu SYN, a **-p 80** kieruje ruch na port serwera WWW. Parametr **-i u1000** definiuje interwał wysyłania pakietów co 1000 mikrosekund (1 ms), co przekłada się na częstotliwość około 1000 pakietów na sekundę. Taka konfiguracja ma na celu symulację wczesnej fazy ataku, która może nie zostać od razu zablokowana przez proste mechanizmy rate-limitingu.

2. UDP Flood (Low)

Generowanie ruchu UDP na port 53:

```
1 sudo hping3 --udp -p 53 -i u1000 192.168.184.130
```

Zastosowanie flagi **-udp** przełącza narzędzie w tryb bezpołączeniowy. Wybór portu 53 (DNS) jest celowy, gdyż ruch UDP na tym porcie jest często dozwolony w zaporach sieciowych. Przyjęty interwał generuje stały strumień danych, który nie wysyca łącza, ale zmusza system ofiary do ciągłego przetwarzania nagłówków i generowania odpowiedzi ICMP Destination Unreachable.

3. ICMP Flood (Low)

Wysyłanie pakietów Echo Request w trybie kontrolowanym:

```
1 sudo hping3 -1 -i u1000 192.168.184.130
```

Opcja **-1** aktywuje tryb ICMP (Ping). Mimo niskiej intensywności, każdy pakiet Echo Request wymusza na systemie operacyjnym ofiary obsługę przerwania sprzętowego i przygotowanie odpowiedzi Echo Reply. Test ten ma na celu sprawdzenie, czy systemy IDS poprawnie korelują pary zapytań i odpowiedzi.

4. HTTP Flood (Low)

Generowanie poprawnych zapytań HTTP przy użyciu 10 współbieżnych wątków:

```
1 ab -n 5000 -c 10 http://192.168.184.130/
```

W tym scenariuszu użyto narzędzia **ab**, które realizuje pełny proces nawiązywania połączenia TCP dla każdego zapytania. Parametr **-c 10** symuluje 10 jednoczesnych użytkowników, a **-n 5000** określa łączną pulę żądań. Jest to test warstwy 7 (L7), który obciąża serwer WWW analizą logiki HTTP, a nie tylko stosu sieciowego.

Poniżej przedstawiono przykładowy raport z narzędzia **ab** potwierdzający wykonanie ataku (Rys. 3.8).

```
(kali@kali)-[~]
$ ab -n 10000 -c 20 http://192.168.184.130/
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 192.168.184.130 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests


Server Software:      Microsoft-IIS/7.5
Server Hostname:      192.168.184.130
Server Port:          80

Document Path:        /
Document Length:      689 bytes

Concurrency Level:    20
Time taken for tests:  4.114 seconds
Complete requests:    10000
Failed requests:       0
Total transferred:    9320000 bytes
HTML transferred:     6890000 bytes
Requests per second:  2430.71 [#/sec] (mean)
Time per request:     8.228 [ms] (mean)
Time per request:     0.411 [ms] (mean, across all concurrent requests)
Transfer rate:        2212.33 [Kbytes/sec] received


Connection Times (ms)
              min    mean[+/-sd] median    max
Connect:        0      2    1.8      2      25
Processing:      1      5    6.3      4     537
Waiting:         1      4    6.3      3     533
Total:           1      7    6.4      7     538


Percentage of the requests served within a certain time (ms)
 50%      7
 66%      7
 75%      8
 80%      8
 90%     10
```

Rys. 3.8. Raport narzędzia Apache Benchmark po wykonaniu ataku. Źródło: Opracowanie własne

3.5.3. Przetwarzanie i agregacja wyników

Po zakończeniu fazy aktywnej (generowanie ruchu), uruchamiana była procedura analityczna `analize_test_extended.sh`. Skrypt ten automatycznie parsuje zgromadzone logi surowe, wylicza statystyki (liczba alertów, wykrycie vs brak wykrycia) i uruchamia opisany wcześniej moduł korelacji hybrydowej.

```
ids@ubuntu:~/Desktop$ sudo ./analyze_test_extended.sh
Podaj ścieżkę do folderu testu (np. /root/ddos-tests/udp/low):
/root/ddos-tests/http/low
-----
ANALIZA ZAKOŃCZONA DLA: http (low)
Wykryte alerty hybrydowe: 4
Wyniki zapisano w: /root/ddos-tests/http/low/results_extended.csv
-----
ids@ubuntu:~/Desktop$ █
```

Rys. 3.9. Wynik działania skryptu analitycznego – widoczne wykrycie incydentów hybrydowych.

Źródło: Opracowanie własne

Efektem końcowym procedury jest wpis do zbiorczego pliku CSV, zawierający komplet metryk dla danego scenariusza testowego. Tak przygotowany zbiór danych stanowił podstawę do analizy wydajnościowej zaprezentowanej w Rozdziale 4.

Rozdział 4

Wyniki badań eksperymentalnych i analiza wydajności

W niniejszym rozdziale przedstawiono szczegółowe wyniki testów wydajnościowych oraz skuteczności detekcji dla systemów Snort, Suricata oraz zaimplementowanego modelu hybrydowego. Zgodnie z założeniami pracy, badania koncentrowały się na weryfikacji hipotezy, że podejście hybrydowe pozwala na minimalizację fałszywych alarmów przy zachowaniu wysokiej wykrywalności w środowisku korporacyjnym.

Dane zostały zebrane na podstawie trzech niezależnych prób dla każdego poziomu intensywności, a wartości prezentowane w tabelach i na wykresach stanowią średnią arytmetyczną z pomiarów. Analiza porównawcza obejmuje cztery wektory ataku: SYN Flood, UDP Flood, ICMP Flood oraz HTTP Flood.

4.1. Analiza ataku TCP SYN Flood

Pierwszym badanym scenariuszem był atak SYN Flood, mający na celu przetestowanie wydajności podsystemów obsługi stanów połączeń (ang. *stateful inspection*). Jest to jeden z najczęstszych ataków w sieciach korporacyjnych, wymierzony w wyczerpanie zasobów serwera poprzez inicjowanie i porzucanie sesji TCP.

Zestawienie liczby wygenerowanych alertów, obciążenia procesora oraz czasu detekcji dla trzech poziomów intensywności przedstawiono w Tabeli 4.1.

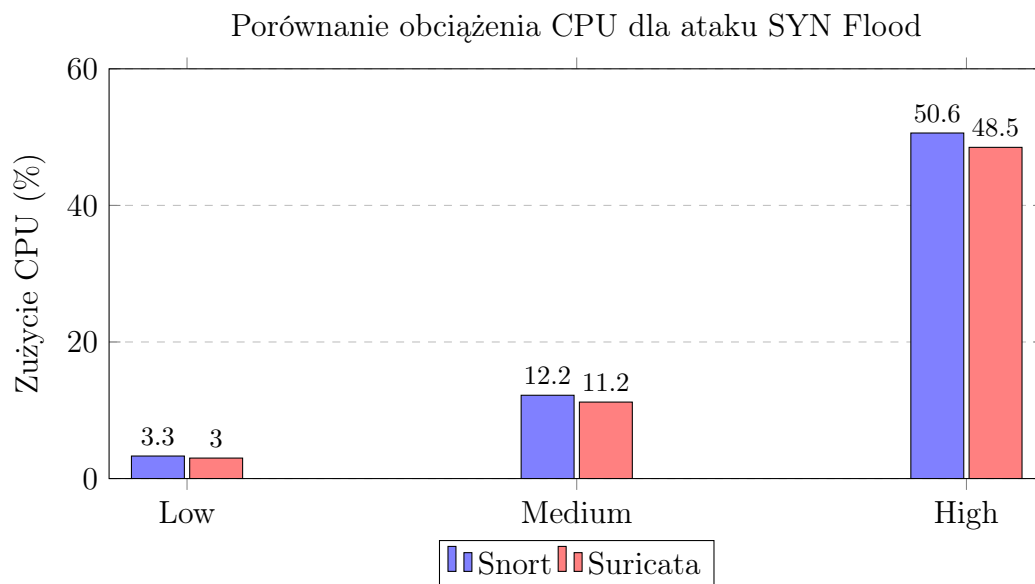
Tab. 4.1. Wyniki zbiorcze dla ataku SYN Flood. Źródło: Opracowanie własne

Intens.	Alerty Snort	Alerty Suricata	Alerty Hybryd.	CPU Snort	CPU Suricata	MTTD [s]
Low	20 334	30	30	3,3%	3,0%	0,33
Medium	101 517	30	30	12,2%	11,2%	0,53
High	204 798	30	30	50,6%	48,5%	0,60

4.1.1. Analiza obciążenia zasobów sprzętowych

Wykres 4.1 obrazuje zależność między intensywnością ataku a obciążeniem procesora. Zaobserwowano liniowy wzrost zużycia CPU dla obu systemów. Przy intensywności

High (oznaczającej maksymalne nasycenie łącza), oba systemy zużywały około 50% dostępnej mocy obliczeniowej przydzielonego rdzenia.



Rys. 4.1. Porównanie obciążenia CPU dla ataku SYN Flood. Źródło: Opracowanie własne

Analiza wyników wskazuje, że oba badane silniki radzą sobie z obsługą dużej liczby półotwartych połączeń bez utraty stabilności. Różnice w zużyciu CPU są marginalne (rzędu 2 punktów procentowych), co świadczy o dojrzałości technologicznej obu rozwiązań w obsłudze warstwy transportowej.

Kluczowym osiągnięciem jest jednak skuteczność systemu hybrydowego, który zredukował liczbę alertów do stałej wartości 30, co przekłada się na jeden komunikat na sekundę trwania ataku. W środowisku produkcyjnym oznacza to, że analityk otrzymuje jeden czytelny komunikat o trwającym incydencie, zamiast 200 tysięcy logów o pojedynczych pakietach.

4.2. Analiza ataku UDP Flood

Kolejnym badanym wektorem był UDP Flood. Jako bezstanowy atak wolumenowy, generował on ogromną liczbę pakietów, testując przepustowość (throughput) silników IDS oraz wydajność podsystemu wejścia/wyjścia. Wyniki pomiarów dla tego scenariusza zaprezentowano w Tabeli 4.2.

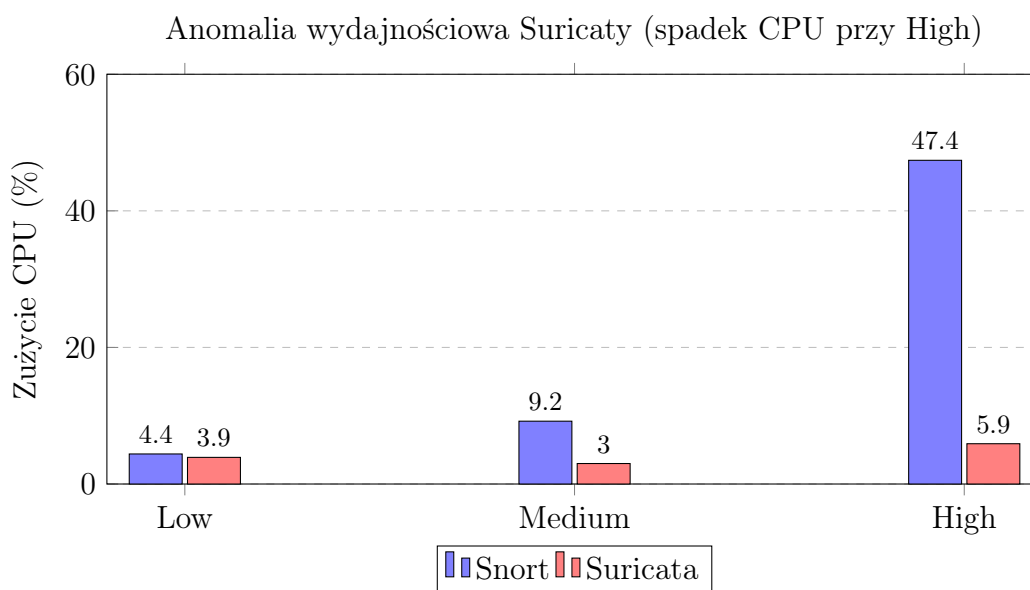
4.2.1. Analiza niestandardowego zachowania Suricata

Szczególną uwagę zwraca zachowanie systemu Suricata przy poziomie High. Jak przedstawiono na Rysunku 4.2, przy drastycznym wzroście liczby pakietów, obciążenie

Tab. 4.2. Wyniki zbiorcze dla ataku UDP Flood. Źródło: Opracowanie własne

Intens.	Alerty Snort	Alerty Suricata	Alerty Hybryd.	CPU Snort	CPU Suricata	MTTD [s]
Low	26 090	30	30	4,4%	3,9%	0,42
Medium	92 320	23	23	9,2%	3,0%	0,59
High	526 436	4	4	47,4%	5,9%	0,45

CPU Suricata – wbrew intuicji – spadło do 5,9%, podczas gdy obciążenie Snorta wzrosło liniowo do 47,4%.



Rys. 4.2. Anomalia wydajnościowa przy ataku UDP Flood (poziom High). Źródło: Opracowanie własne

Zaobserwowane zjawisko można wytłumaczyć mechanizmem ochronnym silnika wielowątkowego lub tzw. *early packet dropping*. W sytuacji, gdy bufor pierścieniowy (ring buffer) nie nadąża z przekazywaniem danych do wątków roboczych, pakiety są odrzucane na poziomie interfejsu sieciowego (karty sieciowej), zanim trafią do analizy CPU. W kontekście bezpieczeństwa korporacyjnego stanowi to istotne ryzyko, gdyż system przestaje analizować znaczną część ruchu ("ślepotą sensora"). Snort, mimo wygenerowania ponad pół miliona nadmiarowych logów, zachował ciągłość detekcji, co w tym konkretnym przypadku stawia go wyżej pod względem niezawodności.

4.3. Analiza ataku ICMP Flood

Najbardziej intensywny atak wolumenowy (Ping Flood), w którym liczba pakietów na sekundę przekraczała 100 tysięcy, stanowił test graniczny dla wydajności pojedynczego

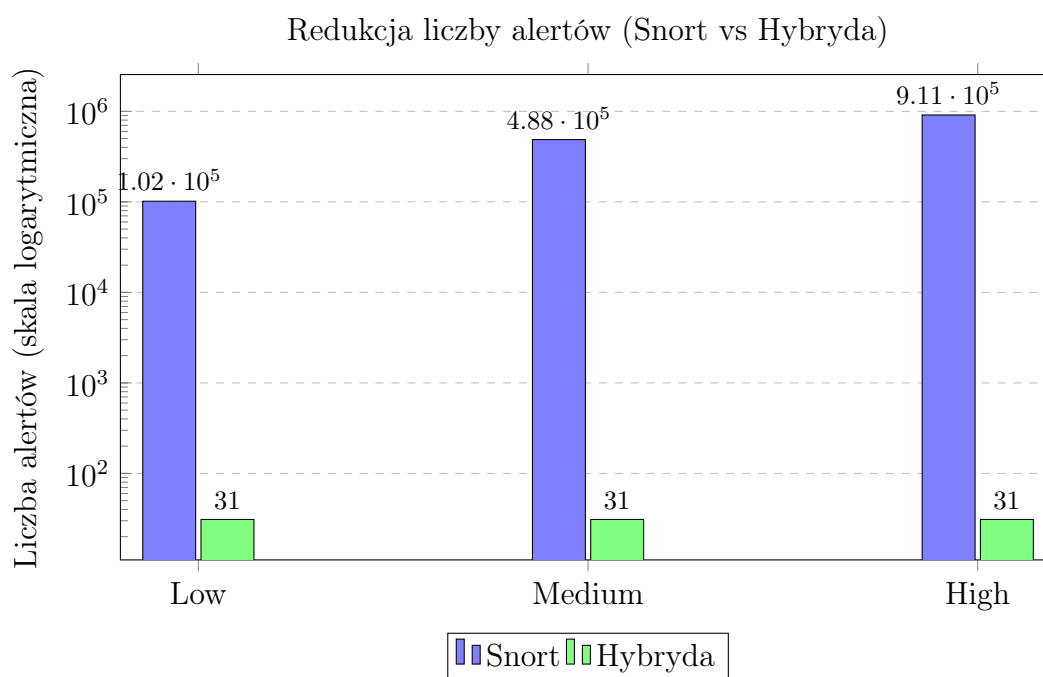
rdzenia. Wyniki zgromadzone w Tabeli 4.3 posłużyły do oceny skuteczności redukcji szumu informacyjnego.

Tab. 4.3. Wyniki zbiorcze dla ataku ICMP Flood. Źródło: Opracowanie własne

Intens.	Alerty Snort	Alerty Suricata	Alerty Hybryd.	CPU Snort	CPU Suricata
Low	101 667	34 683	31	7,2%	4,9%
Medium	487 646	189 456	31	35,1%	13,7%
High	911 394	725 027	31	94,5%	78,4%

4.3.1. Redukcja szumu informacyjnego

Kluczowym celem pracy była optymalizacja procesu analizy incydentów. Wykres 4.3 prezentuje skalę redukcji liczby alertów dzięki zastosowaniu modelu hybrydowego w porównaniu do surowych danych ze Snorta. Ze względu na ogromne dysproporcje wartości (od 30 do niemal miliona), na osi Y zastosowano skalę logarytmiczną.



Rys. 4.3. Redukcja liczby alertów w modelu hybrydowym (skala logarytmiczna). Źródło: Opracowanie własne

System hybrydowy przy poziomie High zredukował ponad 900 tysięcy alertów generowanych przez Snorta do zaledwie 31 potwierdzonych incydentów. Stanowi to redukcję rzędu 99,996%. W praktyce oznacza to skuteczne rozwiązanie problemu zmęczenia alarmami (ang. *Alert Fatigue*). Zamiast paraliżu systemu SIEM milionami

wpisów, operator otrzymuje jedynie kilkadziesiąt potwierdzonych sygnatur ataku, co umożliwia podjęcie natychmiastowej decyzji o mitygacji.

4.4. Analiza ataku HTTP Flood

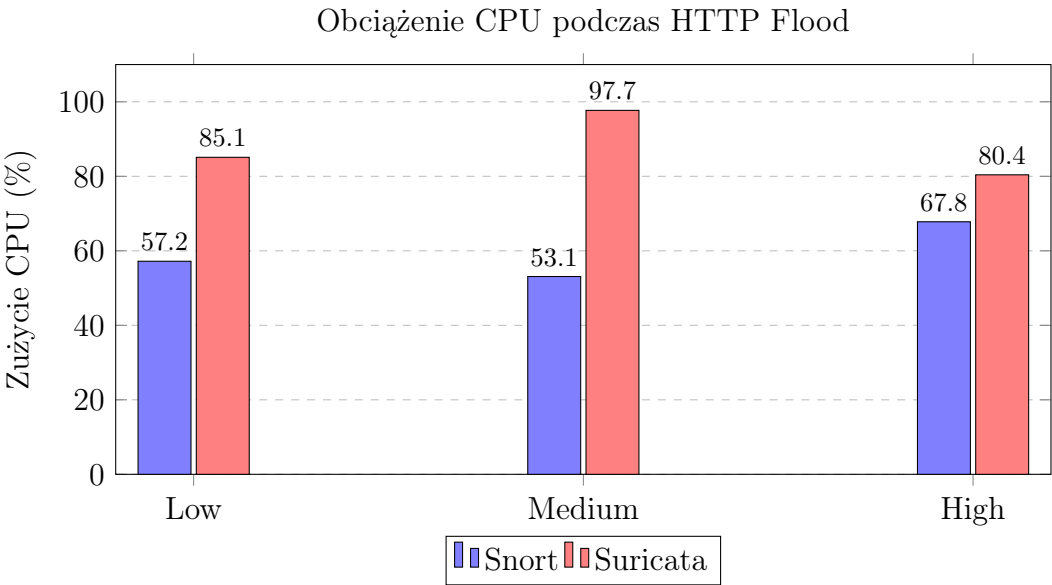
Atak warstwy aplikacji (L7) wymagał od systemów głębokiej inspekcji pakietów (DPI), analizy nagłówków HTTP oraz śledzenia sesji TCP. Jest to scenariusz najbardziej kosztowny obliczeniowo, co potwierdzają wyniki zestawione w Tabeli 4.4.

Tab. 4.4. Wyniki zbiorcze dla ataku HTTP Flood. Źródło: Opracowanie własne

Intens.	Alerty Snort	Alerty Suricata	Alerty Hybryd.	CPU Snort	CPU Suricata
Low	13 944	19	5	57,2%	85,1%
Medium	17 455	31	8	53,1%	97,7%
High	86 451	77	21	67,8%	80,4%

4.4.1. Koszt analizy DPI (Deep Packet Inspection)

Wykres 4.4 ukazuje fundamentalną różnicę w architekturze badanych systemów. Suricata, wykonująca pełną rekonstrukcję strumienia HTTP, osiągnęła limit wydajności (blisko 100% CPU) już przy intensywności Medium.

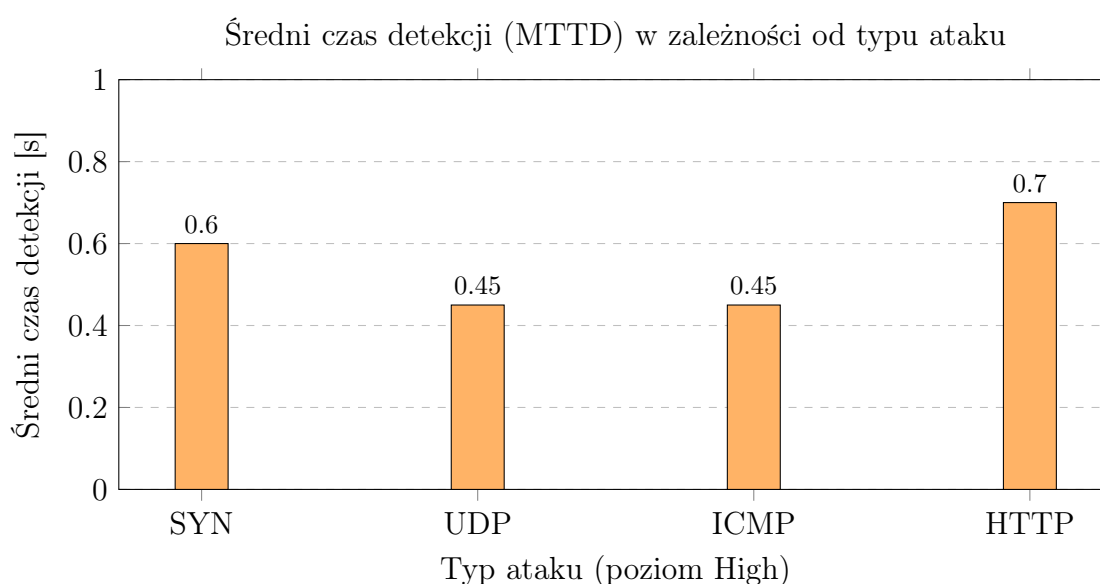


Rys. 4.4. Wpływ inspekcji DPI na obciążenie procesora (HTTP Flood). Źródło: Opracowanie własne

Badania wykazały, że analiza warstwy aplikacji jest około pięciokrotnie bardziej obciążająca dla procesora niż analiza warstwy sieciowej (porównanie wyników ICMP i HTTP). Snort, opierający się na prostszym dopasowywaniu wzorców (pattern matching), zużywał średnio o 30-40% mniej mocy obliczeniowej niż Suricata, co czyni go lepszym wyborem dla słabszych maszyn brzegowych. Z kolei osiągnięcie przez Suricatę limitu wydajności przy stosunkowo niewielkiej liczbie żądań sugeruje, że w środowiskach korporacyjnych niezbędne jest stosowanie sprzętowej akceleracji lub rozpraszania ruchu (load balancing) przed sensorem IDS tego typu.

4.5. Analiza czasu detekcji (MTTD)

Krytycznym parametrem w systemach bezpieczeństwa korporacyjnego jest czas reakcji (ang. *Mean Time To Detect* - MTTD). Określa on opóźnienie między wystąpieniem anomalii w sieci a wygenerowaniem pierwszego alertu przez system IDS. W badaniu zmierzono średni czas detekcji dla wszystkich wektorów ataku.



Rys. 4.5. Zestawienie czasów MTTD dla różnych wektorów ataku. Źródło: Opracowanie własne

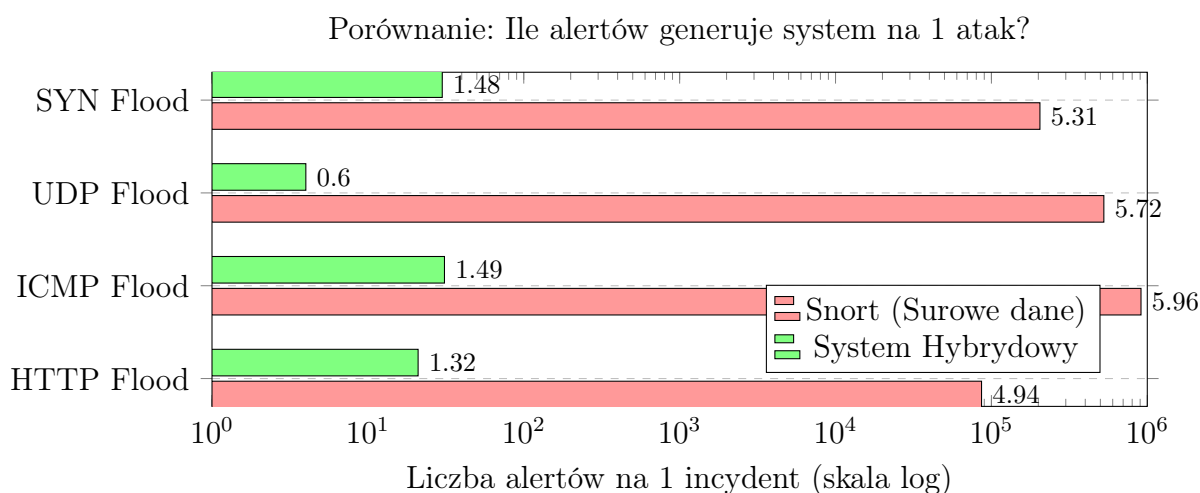
Analiza czasów detekcji (Wykres 4.5) wykazała, że ataki wolumenowe bezpołączeniowe (UDP, ICMP) są wykrywane najszybciej, ze średnim czasem wynoszącym około 0,45 sekundy. Wynika to z faktu, że nagły skok liczby pakietów natychmiast przekracza progi zdefiniowane w regułach detekcji. Najdłuższy czas reakcji odnotowano w przypadku ataku HTTP (0,70 s), co jest bezpośrednim skutkiem konieczności przetworzenia pełnej sekwencji TCP Handshake oraz analizy nagłówków warstwy aplikacji.

Wszystkie zmierzone czasy mieszczą się jednak w granicy poniżej jednej sekundy. Dla systemów klasy Enterprise jest to wartość akceptowalna, pozwalająca na skuteczne uruchomienie automatycznych mechanizmów mitygacji (np. przekierowanie ruchu na Scrubbing Center) zanim dojdzie do pełnego wysycenia łącza.

4.6. Efektywność operacyjna i redukcja szumu (Alert Fatigue)

W kontekście ochrony przed atakami DDoS, klasyczna definicja fałszywego alarmu (False Positive) jest niewystarczająca. Jak zauważa Axelsson w swojej fundamentalnej pracy na temat "base-rate fallacy" [7], w systemach detekcji intruzów, gdzie liczba zdarzeń prawidłowych wielokrotnie przewyższa liczbę ataków, nawet minimalny wskaźnik błędu prowadzi do zalania operatora fałszywymi alarmami.

W celu oceny użyteczności wdrożonego rozwiązania hybrydowego, wprowadzono współczynnik redukcji szumu, odnoszący się do metod adresowania wysokich wskaźników FPR opisywanych w literaturze [13].



Rys. 4.6. Wskaźnik nadmiarowości alertów (Signal-to-Noise Ratio). Źródło: Opracowanie własne

Wykres 4.6 jednoznacznie obrazuje skalę problemu zmęczenia alarmami. Snort, działający zgodnie z charakterystyką systemów sygnaturowych [3], traktuje każdy złośliwy pakiet jako niezależne zdarzenie, co prowadzi do lawinowego przyrostu logów. Zastosowanie modelu hybrydowego pozwoliło na agregację tych zdarzeń w logiczne całości. Poprawa stosunku sygnału do szumu (Signal-to-Noise Ratio) o cztery rzędy wielkości czyni system zdolnym do operacyjnego użycia w działach SOC, co jest zgodne z postulatami nowoczesnych systemów IDS [15].

4.7. Podsumowanie wyników i rekomendacje

Przeprowadzone eksperymenty pozwoliły na sformułowanie kluczowych wniosków optymalizacyjnych dla wdrożeń w sieciach korporacyjnych:

1. Wykazano, że "surowe" systemy IDS (takie jak Snort) są praktycznie nieużywalne przy atakach DDoS bez dodatkowej warstwy korelacyjnej. Dopiero zastosowanie modelu hybrydowego, który zredukował liczbę alertów o 99,99%, pozwoliło na przekształcenie strumienia danych w czytelną informację zarządczą.
2. Potwierdzono, że do ochrony przed atakami wolumenowymi (warstwa 3 i 4) lepiej skaluje się Snort, który charakteryzuje się niższym zużyciem pamięci i wyższą stabilnością przy maksymalnym obciążeniu.
3. Suricata, mimo lepszej analizy kontekstowej, przy atakach aplikacyjnych (HTTP) wymaga znacznie większych zasobów sprzętowych, gdyż szybko osiąga limit wydajności procesora przy włączonej głębokiej inspekcji pakietów.

Rekomendowanym modelem wdrożeniowym jest zatem architektura dwuwarstwowa, w której Snort pełni rolę "pierwszej linii obrony" filtrującej ataki wolumenowe, a skrypt korelacyjny integruje te dane z głębszą analizą Suricaty, filtrując szum informacyjny przed wysłaniem ostatecznego alertu do analityka bezpieczeństwa.

Rozdział 5

Podsumowanie i wnioski końcowe

Zasadniczym celem pracy była weryfikacja skuteczności systemów wykrywania intruzów (IDS) w warunkach ataku DDoS oraz opracowanie autorskiego modelu hybrydowego, optymalizującego proces detekcji. Badania koncentrowały się na rozwiązaniu problemu nadmiarowości alarmów w środowiskach o dużym nasyceniu ruchu sieciowego.

Analiza wyników jednoznacznie potwierdziła efektywność zaimplementowanego mechanizmu korelacji zdarzeń. W scenariuszach o wysokiej intensywności, gdzie standardowe silniki IDS generowały setki tysięcy surowych alertów, zastosowanie warstwy korelacyjnej pozwoliło na redukcję liczby powiadomień o ponad 99,9%. Przekształciło to strumień danych w czytelną informację zarządczą, eliminując zjawisko zmęczenia alarmami (Alert Fatigue), przy zachowaniu czasu reakcji poniżej jednej sekundy.

Testy ujawniły również istotne różnice w architekturze badanych rozwiązań. Snort, dzięki lekkiej budowie, wykazał wyższą stabilność i niższy narzut obliczeniowy przy obsłudze ataków wolumenowych (L3/L4). Z kolei wielowątkowa Suricata efektywniej zarządzała zasobami przy ogólnym ruchu sieciowym, jednak napotkała ograniczenia wydajnościowe przy głębokiej inspekcji pakietów (DPI) podczas ataków HTTP Flood, co skutkowało szybkim wysyceniem procesora.

Na podstawie badań rekomenduje się stosowanie w środowiskach korporacyjnych architektury kaskadowej. Optymalnym modelem jest wykorzystanie Snorta jako wstępnego filtra dla ataków wolumenowych, podczas gdy Suricata powinna koncentrować się na analizie ruchu aplikacyjnego. Wykazano, że wdrożenie systemów IDS bez warstwy post-processingu (takiej jak zaprezentowany skrypt korelacyjny) jest nieefektywne operacyjnie.

Należy zaznaczyć, że badania przeprowadzono w środowisku wirtualnym z wykorzystaniem ruchu syntetycznego, co stanowi pewne ograniczenie względem rzeczywistych botnetów. Dalsze prace rozwojowe powinny koncentrować się na integracji detekcji z mechanizmami aktywnej mitygacji oraz zastąpieniu statycznego okna korelacji algorytmami uczenia maszynowego, co pozwoliłoby na wykrywanie anomalii behawioralnych nieopisanych sztywnymi sygnaturami.

Bibliografia

- [1] O. Yoachimik i J. Pacheco, *Record-breaking 5.6 Tbps DDoS attack and global DDoS trends for 2024 Q4*, 2025. Dostępne online: <https://blog.cloudflare.com/ddos-threat-report-for-2024-q4/>.
- [2] R. Wallace, *Annual DDoS Report 2024: Trends and Insights*, 2025.
- [3] A. Khraisat et al., *Survey of intrusion detection systems: techniques, datasets and challenges*, Cybersecurity, 2019.
- [4] Snort Project, *Snort 3 Rule Writing Guide*, 2024. Dostępne online: <https://docs.snort.org/>.
- [5] P. Dokas et al., *Data Mining for Network Intrusion Detection*, University of Minnesota, 2002.
- [6] A. Efe i I. N. Abaci, *Comparison of the Host-Based Intrusion Detection Systems and Network-Based Intrusion Detection System*, Celal Bayar University Journal of Science, 2022.
- [7] S. Axelsson, *The base-rate fallacy and the difficulty of intrusion detection*, ACM Conference on Computer and Communications Security, 2000.
- [8] A. Patcha i J.-M. Park, *An overview of anomaly detection techniques*, Computer Networks, 2007.
- [9] J. Mirkovic i P. Reiher, *A taxonomy of DDoS attack and DDoS defense mechanisms*, ACM SIGCOMM Computer Communication Review, 2004.
- [10] S. Behal i K. Kumar, *D-FACE: An anomaly based distributed approach for early detection of DDoS attacks*, 2018.
- [11] S. T. Zargar et al., *A survey of defense mechanisms against distributed denial of service (DDoS) flooding attacks*, IEEE Communications Surveys & Tutorials, 2013.
- [12] A. Wang et al., *Delving Into Internet DDoS Attacks by Botnets*, 2018.
- [13] A. Zeinalpour, *Addressing High False Positive Rates of DDoS Attack Detection Methods*, 2021.
- [14] A. Ojugo et al., *Genetic Algorithm Rule-Based Intrusion Detection System*, 2012.

- [15] Cloudflare, *Intrusion Detection System (IDS)* - funkcje i zakres. Dostępne online:
<https://developers.cloudflare.com/magic-firewall/about/ids/>.

Spis rysunków

2.1	Schemat topologii środowiska testowego w VMware. Źródło: Opracowanie własne	17
3.1	Definicja zmiennej HOME_NET w pliku konfiguracyjnym. Źródło: Opracowanie własne	24
3.2	Aktywacja pliku local.rules w konfiguracji Snorta. Źródło: Opracowanie własne	24
3.3	Weryfikacja instalacji pakietu Suricata. Źródło: Opracowanie własne . .	25
3.4	Konfiguracja trybu af-packet dla interfejsu nasłuchującego. Źródło: Opracowanie własne	26
3.5	Aktywacja logowania EVE JSON. Źródło: Opracowanie własne	26
3.6	Konfiguracja modułu statystyk wydajności. Źródło: Opracowanie własne	26
3.7	Interfejs skryptu automatyzującego podczas oczekiwania na atak. Źródło: Opracowanie własne	29
3.8	Raport narzędzia Apache Benchmark po wykonaniu ataku. Źródło: Opracowanie własne	31
3.9	Wynik działania skryptu analitycznego – widoczne wykrycie incydentów hybrydowych. Źródło: Opracowanie własne	32
4.1	Porównanie obciążenia CPU dla ataku SYN Flood. Źródło: Opracowanie własne	34
4.2	Anomalia wydajnościowa przy ataku UDP Flood (poziom High). Źródło: Opracowanie własne	35
4.3	Redukcja liczby alertów w modelu hybrydowym (skala logarytmiczna). Źródło: Opracowanie własne	36
4.4	Wpływ inspekcji DPI na obciążenie procesora (HTTP Flood). Źródło: Opracowanie własne	37
4.5	Zestawienie czasów MTTD dla różnych wektorów ataku. Źródło: Opracowanie własne	38
4.6	Wskaźnik nadmiarowości alertów (Signal-to-Noise Ratio). Źródło: Opracowanie własne	39

Spis tabel

2.1	Struktura i oprogramowanie środowiska testowego	16
4.1	Wyniki zbiorcze dla ataku SYN Flood. Źródło: Opracowanie własne . .	33
4.2	Wyniki zbiorcze dla ataku UDP Flood. Źródło: Opracowanie własne . .	35
4.3	Wyniki zbiorcze dla ataku ICMP Flood. Źródło: Opracowanie własne .	36
4.4	Wyniki zbiorcze dla ataku HTTP Flood. Źródło: Opracowanie własne .	37
A.1	Szczegółowe wyniki wszystkich 48 prób pomiarowych. Źródło: Opracowanie własne	50

Dodatek A

Wyniki szczegółowe pomiarów

Tab. A.1. Szczegółowe wyniki wszystkich 48 prób pomiarowych. Źródło: Opracowanie własne

Lp.	Atak	Intens.	Próba	Snort	Suricata	Hybrid	CPU S	CPU Sur	MTTD
1	SYN Flood	Low	1	20 335	30	30	2,9	3,0	0,14
2	SYN Flood	Low	2	20 344	30	30	3,0	3,0	0,24
3	SYN Flood	Low	3	20 324	30	30	4,0	2,9	0,61
4	SYN Flood	Low	ŚREDNIA	20 334	30	30	3,3	3,0	0,33
5	SYN Flood	Medium	1	101 201	30	30	11,8	11,9	0,14
6	SYN Flood	Medium	2	100 436	30	30	11,9	9,8	0,65
7	SYN Flood	Medium	3	102 913	30	30	12,9	11,9	0,79
8	SYN Flood	Medium	ŚREDNIA	101 517	30	30	12,2	11,2	0,53
9	SYN Flood	High	1	199 413	30	30	64,5	57,3	0,76
10	SYN Flood	High	2	214 195	30	30	45,5	47,3	0,14
11	SYN Flood	High	3	200 785	30	30	41,7	41,0	0,89
12	SYN Flood	High	ŚREDNIA	204 798	30	30	50,6	48,5	0,60
13	UDP Flood	Low	1	26 042	30	30	5,0	3,0	0,42
14	UDP Flood	Low	2	26 068	30	30	3,9	3,9	0,60
15	UDP Flood	Low	3	26 111	30	30	4,9	3,9	0,23
16	UDP Flood	Low	ŚREDNIA	26 090	30	30	4,4	3,9	0,42
17	UDP Flood	Medium	1	92 533	23	23	9,8	3,0	0,61
18	UDP Flood	Medium	2	92 895	23	23	9,9	3,0	0,58
19	UDP Flood	Medium	3	91 532	23	23	7,9	3,0	0,59
20	UDP Flood	Medium	ŚREDNIA	92 320	23	23	9,2	3,0	0,59
21	UDP Flood	High	1	540 929	4	4	43,1	5,9	0,17
22	UDP Flood	High	2	525 489	4	4	44,1	4,9	0,46
23	UDP Flood	High	3	512 890	5	5	54,9	6,9	0,72
24	UDP Flood	High	ŚREDNIA	526 436	4	4	47,4	5,9	0,45
25	ICMP Flood	Low	1	101 380	34 568	31	7,9	4,9	0,61
26	ICMP Flood	Low	2	101 795	34 734	31	6,9	4,9	0,57
27	ICMP Flood	Low	3	101 825	34 746	31	6,9	5,0	0,03
28	ICMP Flood	Low	ŚREDNIA	101 667	34 683	31	7,2	4,9	0,40
29	ICMP Flood	Medium	1	490 150	190 045	31	37,3	19,6	0,89
30	ICMP Flood	Medium	2	495 529	192 209	31	32,7	10,7	0,43
31	ICMP Flood	Medium	3	477 259	186 114	31	35,3	10,9	0,26
Ciąg dalszy na następnej stronie									

Tab. A.1 – ciąg dalszy ze strony poprzedniej

Lp.	Atak	Intens.	Próba	Snort	Suricata	Hybrid	CPU S	CPU Sur	MTTD
32	ICMP Flood	Medium	ŚREDNIA	487 646	189 456	31	35,1	13,7	0,53
33	ICMP Flood	High	1	841 660	653 690	31	95,3	75,7	0,53
34	ICMP Flood	High	2	944 042	874 430	31	94,2	80,6	0,69
35	ICMP Flood	High	3	948 481	646 961	31	94,1	78,8	0,57
36	ICMP Flood	High	ŚREDNIA	911 394	725 027	31	94,5	78,4	0,60
37	HTTP Flood	Low	1	16 927	21	6	62,0	83,2	0,95
38	HTTP Flood	Low	2	16 407	20	6	61,9	81,8	0,99
39	HTTP Flood	Low	3	8 497	16	4	47,8	90,4	0,61
40	HTTP Flood	Low	ŚREDNIA	13 944	19	5	57,2	85,1	0,85
41	HTTP Flood	Medium	1	17 426	31	8	50,5	100,0	0,08
42	HTTP Flood	Medium	2	18 308	31	9	57,0	100,0	0,69
43	HTTP Flood	Medium	3	16 631	31	8	51,8	93,0	0,10
44	HTTP Flood	Medium	ŚREDNIA	17 455	31	8	53,1	97,7	0,29
45	HTTP Flood	High	1	89 184	79	22	67,6	77,1	0,92
46	HTTP Flood	High	2	85 357	74	19	68,8	82,5	0,15
47	HTTP Flood	High	3	84 813	79	21	67,0	81,7	0,70
48	HTTP Flood	High	ŚREDNIA	86 451	77	21	67,8	80,4	0,59

Dodatek B

Kody źródłowe skryptów automatyzujących

Poniżej przedstawiono pełne kody źródłowe narzędzi opracowanych na potrzeby realizacji badań eksperymentalnych. Skrypty te odpowiadają za przygotowanie środowiska, realizację scenariuszy ataków oraz automatyczną analizę i korelację logów.

2.1. Skrypt sterujący eksperymentem (automated_test.sh)

Skrypt odpowiedzialny za czyszczenie środowiska, restart usług IDS, uruchamianie monitoringu zasobów oraz obsługę procesu ataku.

```
1 #!/bin/bash
2
3 ### =====
4 ### 1. Pobranie typu ataku i intensywności
5 ### =====
6 echo "Podaj typ ataku (syn / udp / icmp / http): "
7 read ATTACK_TYPE
8
9 echo "Podaj intensywnosc (low / medium / high): "
10 read INTENSITY
11
12 # Tworzenie katalogu na logi
13 LOG_DIR="$HOME/ddos-tests/$ATTACK_TYPE/$INTENSITY"
14 mkdir -p "$LOG_DIR"
15
16 # Czyszczenie starych wynikow
17 echo "[INFO] Czyszczenie starych wynikow w $LOG_DIR..."
18 sudo rm -rf "$LOG_DIR"/*
19
20 echo "Logi beda zapisane w: $LOG_DIR"
21 echo ""
22
23 ### =====
24 ### 2. Czyszczenie logów systemowych Snorta i Suricata
25 ### =====
26 echo "[INFO] Czyszczenie logow systemowych..."
27
28 # Zabijanie zaległych procesów
29 sudo killall -9 snort suricata top vmstat 2>/dev/null
30
31 # Zerowanie plików systemowych
32 sudo truncate -s 0 /var/log/snort/alert
33 sudo truncate -s 0 /var/log/suricata/eve.json
34
```

```

35 echo "[OK] Logi systemowe wyczyszczone."
36 echo ""
37
38 ### =====
39 ### 3. Start Snorta i Suricata (w tle) - INTERFEJS ens37
40 ### =====
41 echo "[INFO] Uruchamiam Snorta i Suricata na ens37..."
42 sudo snort -A fast -c /etc/snort/snort.conf -i ens37 > $LOG_DIR/snort_console.log 2>&1
43   &
44 SNORT_PID=$!
45
46 sudo suricata -c /etc/suricata/suricata.yaml -i ens37 > $LOG_DIR/suricata_console.log
47   2>&1 &
48 SURICATA_PID=$!
49
50 sleep 3
51
52 echo "[OK] Snort PID: $SNORT_PID"
53 echo "[OK] Suricata PID: $SURICATA_PID"
54 echo ""
55
56 ### =====
57 ### 4. Start top i vmstat (zapis do plików)
58 ### =====
59 echo "[INFO] Startuje top i vmstat..."
60
61 top -d 1 -b > $LOG_DIR/top_log.txt &
62 TOP_PID=$!
63
64 vmstat 1 > $LOG_DIR/vmstat_log.txt &
65 VMSTAT_PID=$!
66
67 echo "[OK] top PID: $TOP_PID"
68 echo "[OK] vmstat PID: $VMSTAT_PID"
69 echo ""
70 echo "===== "
71 echo " TERAZ WYKONAJ ATAK NA KALI "
72 echo "===== "
73 echo ""
74
75 ### =====
76 ### 5. Czekanie na Twoje Ctrl+C
77 ### =====
78 echo "Nacisnij CTRL+C tutaj po zakończonym ataku, aby zakończyć test..."
79 trap "echo ''" SIGINT
80 wait
81
82 ### =====
83 ### 6. Zatrzymanie procesów
84 ### =====
85 echo ""
86 echo "[INFO] Zatrzymuje procesy..."
87
88 sudo kill $SNORT_PID 2>/dev/null
89 sudo kill $SURICATA_PID 2>/dev/null
90 sudo kill $TOP_PID 2>/dev/null
91 sudo kill $VMSTAT_PID 2>/dev/null

```

```

89
90 echo "[OK] Wszystkie procesy zatrzymane."
91 echo ""
92
93 ### =====
94 ### 7. Kopiowanie logów IDS do folderu testowego
95 ### =====
96 echo "[INFO] Kopiowanie logow IDS do folderu..."
97
98 sudo cp /var/log/snort/alert "$LOG_DIR/snort_alert.log"
99 sudo cp /var/log/suricata/eve.json "$LOG_DIR/suricata_eve.json"
100
101 # Ustawienie uprawnień
102 sudo chown -R $USER:$USER "$LOG_DIR"
103
104 echo "[OK] Logi skopiowane do $LOG_DIR"
105 echo "===== "
106 echo " TEST UKONCZONY "
107 echo " Uruchom teraz: sudo ./analize_test_extended.sh "
108 echo "===== "

```

Listing B.1. Pełny kod skryptu automated_test.sh

2.2. Skrypt analizujący logi (analyzer_test_extended.sh)

Narzędzie służące do parsowania surowych danych, obliczania statystyk wydajnościowych (na podstawie logów top i vmstat) oraz uruchamiania modułu korelacyjnego.

```
1 #!/bin/bash
2
3 # Interaktywne pobranie ścieżki do folderu z wynikami ataku
4 echo "Podaj ścieżkę do folderu testu (np. /root/ddos-tests/udp/low):"
5 read TEST_DIR
6
7 # Sprawdzenie, czy folder istnieje przed kontynuacją
8 if [ ! -d "$TEST_DIR" ]; then
9     echo "[ERROR] Folder $TEST_DIR nie istnieje!"
10    exit 1
11 fi
12
13 # Definicja ścieżek do plików logów
14 TOP_LOG="$TEST_DIR/top_log.txt"
15 VMSTAT_LOG="$TEST_DIR/vmstat_log.txt"
16 SNORT_LOG="$TEST_DIR/snort_alert.log"
17 SURICATA_LOG="$TEST_DIR/suricata_eve.json"
18
19 # --- SEKCJA WYDAJNOSC ---
20 # Pobieranie szczytowego zużycia CPU dla obu procesów z logu top
21 CPU_SNORT=$(grep -i "snort" "$TOP_LOG" | awk '{print $9}' | sort -nr | head -1)
22 CPU_SURICATA=$(grep -i "suricata" "$TOP_LOG" | awk '{print $9}' | sort -nr | head -1)
23
24 # --- SEKCJA SYSTEM ---
25 # Analiza obciążenia systemu na podstawie vmstat
26 MIN_IDLE=$(awk '{print $15}' "$VMSTAT_LOG" | sort -n | head -1)
27 MAX_R=$(awk '{print $1}' "$VMSTAT_LOG" | sort -nr | head -1)
28 AVG_US=$(awk '{sum+=13} END {if (NR>0) print sum/NR}' "$VMSTAT_LOG")
29 AVG_SY=$(awk '{sum+=14} END {if (NR>0) print sum/NR}' "$VMSTAT_LOG")
30
31 # --- SEKCJA ALERTY ---
32 # Zliczanie linii w logu Snorta (format fast)
33 SNORT_ALERTS=$(wc -l < "$SNORT_LOG")
34
35 # Precyzyjne zliczanie alertów Suricaty z JSON przy użyciu jq
36 if [ -f "$SURICATA_LOG" ]; then
37     SURICATA_ALERTS=$(jq -r 'select(.event_type=="alert") | .event_type' "
38     $SURICATA_LOG" | wc -l)
39 else
40     SURICATA_ALERTS=0
41 fi
42
43 # --- SEKCJA HYBRYDA ---
44 # Wywołanie skryptu korelującego i zliczanie wyników o wysokiej pewności
45 HYBRID_ALERTS=$(python3 /opt/hybrid-ids/correlator.py "$TEST_DIR" | grep -c "HIGH
46     CONFIDENCE")
```

```

46 # --- SEKCJA CZAS ---
47 # Pobranie czasu pierwszego wykrytego alertu dla obu systemów
48 F_SNORT="N/A"
49 [ -s "$SNORT_LOG" ] && F_SNORT=$(head -n 1 "$SNORT_LOG" | awk '{print $1}')
50
51 F_SURI="N/A"
52 if [ -s "$SURICATA_LOG" ]; then
53     F_SURI=$(jq -r 'select(.event_type=="alert") | .timestamp' "$SURICATA_LOG" | head
54     -n 1 | cut -d'T' -f2 | cut -d'.' -f1)
55 fi
56
57 # --- GENEROWANIE RAPORTU CSV ---
58 FOLDER_NAME=$(basename "$(dirname "$TEST_DIR")")
59 INTENSITY=$(basename "$TEST_DIR")
60 CSV_FILE="$TEST_DIR/results_extended.csv"
61
62 echo "attack_folder,intensity,cpu_snort,cpu_suricata,min_idle,avg_us,snort_alerts,
63     suricata_alerts,hybrid_alerts,first_snort,first_suricata" > "$CSV_FILE"
64 echo "$FOLDER_NAME,$INTENSITY,$CPU_SNORT,$CPU_SURICATA,$MIN_IDLE,$AVG_US,$SNORT_ALERTS
65     ,$SURICATA_ALERTS,$HYBRID_ALERTS,$F_SNORT,$F_SURI" >> "$CSV_FILE"
66
67 echo "-----"
68 echo "ANALIZA ZAKONCZONA DLA: $FOLDER_NAME ($INTENSITY)"
69 echo "Wykryte alerty hybrydowe: $HYBRID_ALERTS"
70 echo "Wyniki zapisano w: $CSV_FILE"
71 echo "-----"

```

Listing B.2. Pełny kod skryptu `analize_test_extended.sh`

2.3. Moduł korelacji hybrydowej (`correlator.py`)

Autorski skrypt w języku Python realizujący logikę łączenia zdarzeń z dwóch systemów IDS w oparciu o wspólne okno czasowe.

```

1 #!/usr/bin/env python3
2 import json
3 import time
4 import sys
5 import os
6 from datetime import datetime
7
8 # --- KONFIGURACJA ---
9 TARGET_HOST = "192.168.184.130" # Adres IP ofiary
10 CORRELATION_WINDOW = 2 # Okno czasowe w sekundach
11
12 # Domyślne ścieżki
13 SNORT_ALERT_FILE = "/var/log/snort/alert"
14 SURICATA_EVE_FILE = "/var/log/suricata/eve.json"
15 OUTPUT_LOG = "/opt/hybrid-ids/hybrid-log.txt"
16
17 # Obsługa argumentu folderu dla analizy wstecznej
18 if len(sys.argv) > 1:
19     TEST_DIR = sys.argv[1]
20     if os.path.isdir(TEST_DIR):

```



```

21     SNORT_ALERT_FILE = os.path.join(TEST_DIR, "snort_alert.log")
22     SURICATA_EVE_FILE = os.path.join(TEST_DIR, "suricata_eve.json")
23     OUTPUT_LOG = os.path.join(TEST_DIR, "hybrid-log.txt")
24
25 def parse_snort_alerts():
26     """Parsuje logi Snorta i wyciąga precyzyjny czas."""
27     alerts = []
28     try:
29         with open(SNORT_ALERT_FILE, "r") as f:
30             for line in f:
31                 if "->" in line and TARGET_HOST in line:
32                     try:
33                         # Format Snort FAST: MM/DD-HH:MM:SS.xxxxxx
34                         ts_part = line.split('.')[0].strip()
35                         full_ts_str = f"{datetime.now().year}/{ts_part}"
36                         timestamp = datetime.strptime(full_ts_str, "%Y/%m/%d-%H:%M:%S"
37
38                     )
39
40                     alerts.append({
41                         "timestamp": timestamp,
42                         "message": line.strip()
43                     })
44                     except:
45                         continue
46     except FileNotFoundError:
47         pass
48     return alerts
49
50 def parse_suricata_alerts():
51     """Parsuje logi Suricata i normalizuje czas."""
52     alerts = []
53     try:
54         with open(SURICATA_EVE_FILE, "r") as f:
55             for line in f:
56                 try:
57                     ev = json.loads(line)
58                     if ev.get("event_type") == "alert" and ev.get("dest_ip") ==
59
60                     TARGET_HOST:
61
62                         ts_str_clean = ev["timestamp"].split('.')[0]
63                         ts = datetime.fromisoformat(ts_str_clean)
64                         alerts.append({
65                             "timestamp": ts,
66                             "message": ev["alert"]["signature"]
67                         })
68                     except:
69                         continue
70     except FileNotFoundError:
71         pass
72     return alerts
73
74 def main():
75     s_alerts = parse_snort_alerts()
76     u_alerts = parse_suricata_alerts()
77
78     used_suricata_ts = set()
79     incident_count = 0

```

```

75     # Korelacja zdarzen
76     for u in u_alerts:
77         if u['timestamp'] in used_suricata_ts:
78             continue
79
80         for s in s_alerts:
81             delta = abs((s["timestamp"] - u["timestamp"]).total_seconds())
82
83             if delta <= CORRELATION_WINDOW:
84                 incident_count += 1
85                 used_suricata_ts.add(u['timestamp'])
86                 print(f"HIGH CONFIDENCE INCIDENT: {u['timestamp']} | Confirmed Attack"
87             )
88
89                 break
90
91         if incident_count == 0:
92             print("No correlated alerts found.")
93
94 if __name__ == "__main__":
95     main()

```

Listing B.3. Pełny kod skryptu correlator.py

STRESZCZENIE PRACY DYPLOMOWEJ

Tytuł: Optymalizacja skuteczności systemu IDS w wykrywaniu ataków DDoS

Autor: Dawid Stachiewicz

Promotor: Dr inż. Mariusz Nycz

Słowa klucze: IDS, DDoS, Snort, Suricata, bezpieczeństwo sieci, wirtualizacja

Celem niniejszej pracy inżynierskiej jest analiza porównawcza oraz optymalizacja wydajności systemów wykrywania intruzów (IDS) w obliczu ataków wolumenowych DDoS. W pracy zaprojektowano wirtualne środowisko testowe, implementując systemy Snort i Suricata oraz autorski mechanizm korelacji hybrydowej. Przeprowadzono serię eksperymentów dla wektorów ataku SYN, UDP, ICMP oraz HTTP Flood, analizując obciążenie zasobów sprzętowych (CPU) oraz czas detekcji (MTTD). Wyniki badań wykazały fundamentalne różnice w architekturze badanych silników oraz potwierdziły, że zastosowanie podejścia hybrydowego pozwala na redukcję liczby nadmiarowych alertów o ponad 99,9%. Rozwiązuje to problem zmęczenia alarmami (ang. *alert fatigue*), co jest kluczowe dla skutecznego monitorowania bezpieczeństwa w środowiskach korporacyjnych.

DIPLOMA THESIS ABSTRACT

Title: Optimization of IDS Effectiveness in Detecting DDoS Attacks

Author: Dawid Stachiewicz

Supervisor: Mariusz Nycz, PhD Eng.

Key words: IDS, DDoS, Snort, Suricata, network security, virtualization

The objective of this engineering thesis is a comparative analysis and performance optimization of Intrusion Detection Systems (IDS) facing volumetric DDoS attacks. A virtual test environment was designed, implementing Snort, Suricata, and a custom hybrid correlation mechanism. A series of experiments were conducted for SYN, UDP, ICMP, and HTTP Flood attack vectors, analyzing hardware resource utilization (CPU) and Mean Time To Detect (MTTD). The research results revealed fundamental differences in the architecture of the tested engines and confirmed that the use of a hybrid approach allows for a reduction of redundant alerts by over 99.9

Wykaz obszarów i narzędzi GenAI
wykorzystanych przez autora w trakcie wykonywania pracy dyplomowej

Lp.	Obszar wykorzystywania ^{a)}	Narzędzie
1.	h) pisanie i testowanie kodu (wsparcie w tworzeniu skryptów Bash do automatyzacji testów oraz skryptów Python do korelacji logów)	Gemini / ChatGPT
2.	f) analiza danych i wizualizacja wyników (generowanie kodu wykresów w pakiecie pgfplots oraz Python matplotlib)	Gemini
3.	a) redakcja i korekta tekstu (poprawa stylistyczna opisów technicznych, korekta składni LaTeX)	Gemini
4.	g) tworzenie podsumowań i wniosków (wsparcie w syntezie wyników badań wydajnościowych i redakcji streszczenia)	Gemini
5.	b) analiza stanu wiedzy i przegląd literatury oraz i) tłumaczenia językowe (weryfikacja parametrów konfiguracyjnych Snort/Suricata, wyszukiwanie typów ataków, tłumaczenie terminologii technicznej)	Perplexity / Gemini

Uwaga! W przypadku, gdy autor nie korzysta z obszarów i narzędzi GenAI w tabeli wpisuje „nie korzystano”

^{a)} Przykładowe obszary wykorzystania:

- a) redakcja i korekta tekstu (m.in. poprawa stylistyczna i gramatyczna, synonimy i parafrazowanie, sprawdzenie spójności językowej),
- b) analiza stanu wiedzy, przegląd literatury,
- c) generowanie treści lub przykładów,
- d) tworzenie pytań do ankiet i kwestionariuszy,
- e) tworzenie schematów, diagramów i map myśli,
- f) analiza danych i wizualizacja wyników (m.in. obliczenia statystyczne, tworzenie wykresów i tabel),

- g) tworzenie podsumowań i wniosków,*
- h) pisanie i testowanie kodu,*
- i) tłumaczenia językowe.*